

ĐẠI HỌC QUỐC GIA TP HCM
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO LAB 3

Chủ đề: Advanced MapReduce & Spark Structured APIs

Môn học: Nhập môn Dữ liệu lớn

Sinh viên thực hiện:

23120099 - Lê Xuân Trí

23120180 - Trần Lê Trung Trực

23120166 - Trần Hữu Kim Thành

23120185 - Nguyễn Hồ Anh Tuấn

Giảng viên hướng dẫn:

Thầy Trần Huy Bàn

Thầy Huỳnh Lâm Hải Đăng

Ngày 19 tháng 5 năm 2026

Mục lục

Bảng thuật ngữ	vi
1 Thông tin nhóm & Phân công công việc	1
2 Giới thiệu	1
2.1 Bối cảnh bài lab	1
2.2 Bộ dữ liệu Amazon Sale Report	1
2.3 Môi trường thực thi	2
2.4 Lý do chọn Scala	2
2.5 Tổng quan kết quả đầu ra	3
2.6 Cấu trúc báo cáo	3
3 Task 1.1: Sliding Window – Most Bought Size per State	3
3.1 Phân tích truy vấn	3
3.2 Phân rã thành các bước cơ bản	4
3.3 Lý do phân rã	5
3.4 Chiến lược thực thi & Triển khai mã nguồn	5
3.5 Phân tích chuyên sâu	7
3.5.1 Pseudocode tổng quan	7
3.5.2 Tại sao vòng lặp lồng nhau gây thơ bất khả thi	8
3.5.3 Thiết kế khóa giải quyết vấn đề thế nào	8
3.5.4 Xử lý các ngày trống không có giao dịch	9
3.6 Kết quả thực thi & Xuất dữ liệu	9
3.7 Minh chứng bằng hình ảnh	10
4 Task 1.2: State-level Median Variety per Month	12
4.1 Phân tích truy vấn	12
4.2 Phân rã thành các bước cơ bản	13
4.3 Lý do phân rã	14
4.4 Chiến lược thực thi & Triển khai mã nguồn	14
4.5 Phân tích chuyên sâu	17

4.5.1	Pseudocode tổng quan	17
4.5.2	Tại sao trung vị KHÔNG thể tính bằng Combiner đơn giản	18
4.5.3	Chiến lược tính trung vị trong mô hình MapReduce	18
4.5.4	Cách xác định “kích cỡ $\geq$ XXL” — logic phân tích	19
4.6	Kết quả thực thi & Xuất dữ liệu	19
4.7	Minh chứng bằng hình ảnh	20
5	Task 2.1: Percentage of Cancelled Orders per City	22
5.1	Phân tích truy vấn	22
5.2	Phân rã thành các bước cơ bản	23
5.3	Lý do phân rã	25
5.4	Chiến lược thực thi & Triển khai mã nguồn	26
5.5	Phân tích chuyên sâu — Kế hoạch thực thi	29
5.5.1	Đầu ra của lệnh explain(true)	29
5.5.2	Phân tích chiến lược liên kết vật lý	29
5.5.3	Đếm các nút trao đổi dữ liệu	30
5.5.4	Đếm các giai đoạn thực thi	31
5.5.5	Thảo luận về cơ chế tự thích ứng khi thực thi truy vấn	32
5.6	Kết quả thực thi & Xuất dữ liệu	32
5.7	Minh chứng bằng hình ảnh	34
6	Task 2.2: StdDev of Amount by SKU-Month with Percentile Threshold	36
6.1	Phân tích truy vấn	36
6.2	Phân rã thành các bước cơ bản	37
6.3	Lý do phân rã	39
6.4	Chiến lược thực thi & Triển khai mã nguồn	40
6.5	Phân tích chuyên sâu	41
6.5.1	Thuật toán bên trong approx_percentile	41
6.5.2	Phân vị chính xác: sắp xếp + đánh số dòng	42
6.5.3	Khi nào phương pháp gần đúng đủ tốt, khi nào bắt buộc dùng chính xác	43
6.6	So sánh hai phương pháp Percentile	43
6.7	Thảo luận về sự phân mảnh dữ liệu và lệch dữ liệu	44

6.8	Kết quả thực thi & Xuất dữ liệu	45
6.9	Minh chứng bằng hình ảnh	46
7	Kết luận	49
7.1	Tổng kết kết quả	49
7.2	Trade-off giữa RDD MapReduce và Spark Structured APIs	50
7.3	Bài học rút ra	51
7.4	Khó khăn và hướng cải thiện	51
8	Phụ lục	52
8.1	Source code và Demo	52

Danh sách bảng

1	Thông tin thành viên nhóm và phân công nhiệm vụ thực hiện Lab 3	1
2	Các trường dữ liệu chính từ Amazon Sale Report dùng trong báo cáo	2
3	Thông số cấu hình môi trường thực thi chương trình	2
4	Tổng quan về các task thực hiện và định dạng tệp đầu ra tương ứng	3
5	Định nghĩa schema tệp đầu ra Task_1-1.csv	4
6	Thống kê tổng số lượng (Qty) theo kích cỡ trong cửa sổ quá khứ tại bang Andaman & Nicobar vào ngày 04-09-22	10
7	Định nghĩa schema tệp đầu ra Task_1-2.csv	12
8	Dữ liệu đơn hàng gốc tại bang Chandigarh vào tháng 03-2022 để kiểm chứng	20
9	Định nghĩa schema tệp đầu ra Task_2-1.parquet	23
10	Chi tiết các phép liên kết trong kế hoạch thực thi vật lý	30
11	Phân tích các nút trao đổi dữ liệu và mục đích sử dụng	31
12	Các chỉ số thống kê tổng quan trích xuất từ tệp Parquet đầu ra của Task 2.1	33
13	Định nghĩa schema tệp đầu ra Task_2-2.parquet	37
14	So sánh tiêu chí sử dụng giữa phương pháp phân vị gần đúng và phân vị chính xác	43
15	Kết quả đo hiệu năng thời gian chạy giữa hai phương pháp tính phân vị	43
16	So sánh mức độ sai lệch kết quả giữa phương pháp gần đúng và chính xác	44
17	Thống kê các nhóm SKU-tháng có số lượng đơn hàng lớn nhất	45
18	So sánh các tiêu chí kỹ thuật và hiệu quả lập trình giữa MapReduce RDD và Spark Structured APIs	50

Danh sách hình vẽ

1	Kết quả chạy chương trình Task 1.1	10
2	Một phần nội dung file <code>Task_1-1.csv</code>	11
3	Kiểm chứng thủ công sliding window cho một state-date mẫu	11
4	Kết quả chạy chương trình Task 1.2	20
5	Một phần nội dung file <code>Task_1-2.csv</code>	21
6	Kiểm chứng thủ công median variety cho một state-month mẫu	22
7	Kết quả chạy chương trình Task 2.1	34
8	Kế hoạch thực thi vật lý (Execution plan) của Task 2.1	35
9	Một phần nội dung tệp Parquet kết quả đầu ra <code>Task_2-1.parquet</code>	36
10	Kết quả chạy chương trình Task 2.2	47
11	Kết quả đo hiệu năng thời gian chạy giữa hai phương pháp	48
12	Một phần nội dung tệp Parquet kết quả đầu ra <code>Task_2-2.parquet</code>	49

Bảng thuật ngữ

Thuật ngữ / Ký hiệu	Ý nghĩa / Định nghĩa
API	Application Programming Interface (Giao diện lập trình ứng dụng)
AQE	Adaptive Query Execution (Cơ chế tự động tối ưu hóa kế hoạch truy vấn khi đang chạy của Spark)
CSV	Comma-Separated Values (Định dạng file văn bản trong đó các giá trị được phân tách bằng dấu phẩy)
DataFrame	Cấu trúc dữ liệu dạng bảng được tối ưu hóa trong Spark Structured APIs
GK	Greenwald-Khanna (Thuật toán nén dữ liệu dùng để ước lượng các giá trị trung vị hoặc percentile một cách gần đúng)
MapReduce	Mô hình lập trình phân tán gồm hai pha chính: Map (ánh xạ dữ liệu) và Reduce (gom nhóm/thu gọn dữ liệu)
Percentile	Phân vị (ngưỡng giá trị chia tập dữ liệu thành các phần có tỷ lệ phần trăm tương ứng)
RDD	Resilient Distributed Dataset (Tập dữ liệu phân tán có khả năng phục hồi - mức trừu tượng cơ bản nhất của Spark)
SKU	Stock Keeping Unit (Mã định danh duy nhất cho từng mặt hàng cụ thể được lưu kho)
Spark Structured API	Các API cấp cao của Spark như DataFrame/Dataset và Spark SQL giúp tự động tối ưu hóa truy vấn
StdDev (stddev_pop)	Population Standard Deviation (Độ lệch chuẩn tổng thể của một tập dữ liệu)
Style	Mã kiểu dáng của sản phẩm (một style có thể có nhiều SKU ứng với các kích cỡ khác nhau)
Sliding Window	Cửa sổ trượt (Kỹ thuật tính toán trên một khoảng thời gian động có độ dài cố định dịch chuyển liên tục)
UDF	User Defined Function (Hàm tự định nghĩa bởi lập trình viên để xử lý các logic phức tạp trong Spark)
Variety	Số lượng SKU phân biệt (distinct) thuộc về cùng một style của sản phẩm

1 Thông tin nhóm & Phân công công việc

MSSV	Họ và Tên	Vai trò	Công việc	Mức độ hoàn thành
23120099	Lê Xuân Trí	Nhóm trưởng	Thực hiện Task 1-1 (MapReduce) & Tổng hợp báo cáo	100%
23120180	Trần Lê Trung Trực	Thành viên	Thực hiện Task 1-2 (MapReduce)	100%
23120166	Trần Hữu Kim Thành	Thành viên	Thực hiện Task 2-1 (Spark Structured API)	100%
23120185	Nguyễn Hồ Anh Tuấn	Thành viên	Thực hiện Task 2-2 (Spark Structured API)	100%

Bảng 1: Thông tin thành viên nhóm và phân công nhiệm vụ thực hiện Lab 3

2 Giới thiệu

2.1 Bối cảnh bài lab

Lab 03 của môn Nhập môn Dữ liệu lớn yêu cầu sinh viên giải quyết bốn bài toán phân tích dữ liệu phân tán trên bộ dữ liệu **Amazon Sale Report**, sử dụng hai nhóm công nghệ:

- **Task 1-1 và Task 1-2:** MapReduce nâng cao — triển khai thuật toán bằng các phép biến đổi Spark RDD theo tư duy xử lý cặp khóa - giá trị.
- **Task 2-1 và Task 2-2:** Spark Structured APIs — sử dụng DataFrame/Dataset API với bộ tối ưu hóa Catalyst.

2.2 Bộ dữ liệu Amazon Sale Report

Bộ dữ liệu chứa thông tin đơn hàng trên nền tảng Amazon tại thị trường Ấn Độ, với khoảng **128,000 dòng** và **24 cột**. Các cột chính được sử dụng trong bốn task:

Cột	Kiểu	Mô tả
Date	String	Ngày đặt hàng, định dạng MM-dd-yy
Status	String	Trạng thái đơn (Shipped, Cancelled, ...)
Fulfilment	String	Đơn vị thực hiện (Amazon, Merchant)
ship-service-level	String	Mức dịch vụ (Standard, Expedited)
ship-city	String	Thành phố giao hàng
ship-state	String	Bang giao hàng
SKU	String	Mã sản phẩm
Style	String	Mã kiểu dáng sản phẩm
Size	String	Size sản phẩm (S, M, L, XL, XXL, ...)
Qty	Integer	Số lượng mua
Amount	Double	Giá trị đơn hàng (INR)
promotion-ids	String	Danh sách mã khuyến mãi
Courier Status	String	Trạng thái vận chuyển

Bảng 2: Các trường dữ liệu chính từ Amazon Sale Report dùng trong báo cáo

2.3 Môi trường thực thi

Thành phần	Chi tiết
Apache Spark	3.4.0
Scala	2.13.8
Chế độ chạy	Chế độ chạy cục bộ (local[*])
Hệ điều hành	Linux chạy trực tiếp trên máy cá nhân
Java	OpenJDK 8 (1.8.0_482)

Bảng 3: Thông số cấu hình môi trường thực thi chương trình

2.4 Lý do chọn Scala

Scala được chọn làm ngôn ngữ triển khai cho toàn bộ bốn task vì ba lý do chính:

1. **Tối ưu điểm số ngôn ngữ:** Theo thang điểm đánh giá, sử dụng Scala cho giải pháp hoàn chỉnh được cộng tối đa 0.125 điểm cho mỗi task.
2. **An toàn kiểu dữ liệu:** Scala là ngôn ngữ định kiểu tĩnh, giúp phát hiện lỗi trong quá trình biên dịch thay vì khi chạy chương trình — điều này đặc biệt quan trọng khi làm việc với Dataset API.

3. **API gốc của Spark:** Spark được viết bằng Scala; các API như `aggregateByKey`, `reduceByKey`, hay các bộ mã hóa của Dataset hoạt động tự nhiên nhất với Scala, tránh được chi phí hiệu năng dư thừa do quá trình tuần tự hóa của PySpark.

2.5 Tổng quan kết quả đầu ra

Task	Công nghệ	Tệp đầu ra	Trạng thái
Task 1.1	MapReduce (RDD)	<code>Task_1-1.csv</code>	Hoàn thành, đã kiểm chứng
Task 1.2	MapReduce (RDD)	<code>Task_1-2.csv</code>	Hoàn thành, đã kiểm chứng
Task 2.1	Spark Structured API	<code>Task_2-1.parquet</code>	Hoàn thành, đã kiểm chứng
Task 2.2	Spark Structured API	<code>Task_2-2.parquet</code>	Hoàn thành, đã kiểm chứng

Bảng 4: Tổng quan về các task thực hiện và định dạng tệp đầu ra tương ứng

2.6 Cấu trúc báo cáo

Báo cáo được tổ chức thành các phần sau: thông tin nhóm (Chương 1), sau đó mỗi task là một chương riêng biệt (Chương 2–5) với cấu trúc thống nhất gồm: phân tích truy vấn, phân rã bước, lý do phân rã, chiến lược thực thi kèm các đoạn mã nguồn minh họa, phân tích chuyên sâu, kết quả và kiểm chứng. Phần kết luận (Chương 6) tổng kết sự cân nhắc đánh giá giữa MapReduce và Spark Structured APIs.

3 Task 1.1: Sliding Window – Most Bought Size per State

3.1 Phân tích truy vấn

Bài toán yêu cầu xác định kích cỡ được mua nhiều nhất tại từng bang/tỉnh theo từng ngày tính toán d . Với mỗi ngày d , cửa sổ thời gian chỉ xét tối đa 7 ngày trước đó, tức $[d - 7, d - 1]$, không bao gồm ngày hiện tại. Vì cửa sổ trượt từng ngày nên kết quả đầu ra có thể chứa các ngày không có đơn hàng trực tiếp trong dữ liệu gốc, kể cả các ngày ngay sau ngày đơn hàng cuối cùng nếu cửa sổ quá khứ vẫn còn dữ liệu đóng góp.

Một dòng dữ liệu được xem là “được mua” khi thỏa đồng thời hai điều kiện: cột **Status** chứa chuỗi **shipped** sau khi chuẩn hóa chữ hoa/thường, và cột **Qty** là số nguyên dương. Cách hiểu này bao phủ

các trạng thái như `Shipped`, `Shipped - Delivered to Buyer`, `Shipped - Returned to Seller`, đúng với mô tả trong đề bài là trạng thái đơn hàng có chứa “shipped”.

Đơn vị đo “mua nhiều nhất” trong lời giải là tổng số lượng (`Qty`) theo kích cỡ trong cửa sổ, không chỉ đơn thuần là số dòng đơn hàng. Với mỗi khóa (`ship-state`, `date`), chương trình so sánh tổng số lượng của từng kích cỡ và chọn kích cỡ có tổng lớn nhất. Nếu nhiều kích cỡ có cùng tổng số lượng, chương trình phá hòa bằng thứ tự từ điển tăng dần của chuỗi kích cỡ để kết quả ổn định và tái lập được.

Kết quả đầu ra của nhiệm vụ này là một tệp CSV thông thường trên hệ thống tệp, có dòng đầu chứa tiêu đề và đúng tên `Task_1-1.csv`. Cấu trúc dữ liệu thực tế:

Cột	Kiểu dữ liệu	Ý nghĩa
<code>ship-state</code>	String	Bang/tỉnh cần tính toán
<code>date</code>	String	Ngày tính toán d , định dạng <code>MM-dd-yy</code>
<code>most_bought_size</code>	String	Kích cỡ có tổng số lượng cao nhất trong cửa sổ
<code>max_quantity</code>	Integer	Tổng số lượng của kích cỡ thắng cuộc trong cửa sổ

Bảng 5: Định nghĩa schema tệp đầu ra `Task_1-1.csv`

3.2 Phân rã thành các bước cơ bản

Lời giải được triển khai theo tư duy MapReduce trên Spark RDD, gồm các bước sau:

- Đọc và kiểm tra cấu trúc dữ liệu đầu vào.** Chương trình đọc tệp CSV, phân tích dòng đầu và kiểm tra đủ các cột bắt buộc: `Date`, `Status`, `Qty`, `Size`, `ship-state`. Nếu thiếu cột, chương trình dừng với thông báo lỗi rõ ràng.
- Phân tích và chuẩn hóa bản ghi.** Mỗi dòng được phân tích cú pháp bằng bộ phân tích CSV tự viết để xử lý dấu phẩy nằm trong các trường có dấu nháy kép. Ngày được phân tích theo các định dạng phổ biến như `MM-dd-yy`, `M-d-yy`, `MM/dd/yy`, `M/d/yy`; cột `Qty` được chuyển kiểu sang số nguyên dương; cột `ship-state` được loại bỏ khoảng trắng thừa (`trim`) và chuẩn hóa viết hoa để tránh tách cùng một bang thành nhiều khóa khác nhau.
- Lọc các mặt hàng được mua thực tế.** Chỉ giữ các dòng có trạng thái `Status` chứa từ khóa `shipped`, số lượng `Qty` > 0 , đồng thời tên bang `ship-state` và kích cỡ `Size` không bị rỗng. Bước này giúp giảm lượng dữ liệu trung gian trước khi phát sinh các khóa cửa sổ.

4. **Sinh đóng góp cho cửa sổ trượt.** Với mỗi giao dịch mua hợp lệ tại ngày t , chương trình phát ra đóng góp cho các ngày tính toán trong tương lai $d = t + 1, t + 2, \dots, t + 7$. Điều này tương đương với việc giao dịch tại ngày t nằm trong cửa sổ quá khứ của các ngày tính toán d đó.
5. **Cộng tổng số lượng theo khóa (`state`, `day`, `size`).** Các đóng góp cùng bang, cùng ngày tính toán và cùng kích cỡ được cộng dồn bằng phép biến đổi `reduceByKey`.
6. **Chọn kích cỡ thắng cuộc theo khóa (`state`, `day`).** Kết quả ở bước trước được đổi khóa thành (`state`, `day`), sau đó bộ giảm chọn kích cỡ có tổng số lượng lớn nhất; nếu xảy ra hòa thì chọn kích cỡ có thứ tự từ điển nhỏ hơn.
7. **Sắp xếp và ghi ra tệp CSV duy nhất.** Kết quả được sắp xếp theo bang `state` và ngày `day`, thêm dòng tiêu đề, dùng `coalesce(1)` để ghi ra một thư mục tạm, sau đó đổi tên tệp bộ phận thành tệp CSV duy nhất có tên chính xác theo yêu cầu.

3.3 Lý do phân rã

Phần khó nhất của bài toán là cơ chế cửa sổ trượt. Nếu tính trực tiếp cho từng ngày d rồi quét ngược lại 7 ngày trước đó, chi phí quét dữ liệu sẽ lặp lại nhiều lần. Lời giải chọn hướng tiếp cận ngược lại: một đơn hàng tại ngày t tự phát sinh đóng góp cho tối đa 7 ngày tương lai mà nó có thể thuộc về cửa sổ. Nhờ vậy, cửa sổ trượt được chuyển đổi thành bài toán gom nhóm khóa - giá trị thông thường.

Cách phân rã này hoàn toàn phù hợp với MapReduce vì mỗi bước đều có khóa rõ ràng. Bước ánh xạ sinh khóa (`state`, `day`, `size`); bước giảm đầu tiên cộng số lượng; bước reduce thứ hai chọn kích cỡ thắng cuộc cho từng khóa (`state`, `day`). Việc lọc dữ liệu hợp lệ trước khi phát sinh 7 đóng góp cũng giúp giảm thiểu số lượng bản ghi trung gian và giảm chi phí truyền tải dữ liệu qua mạng.

3.4 Chiến lược thực thi & Triển khai mã nguồn

Task 1.1 được cài đặt trong file `src/Task_1-1/source/Task1_1.scala`. Chương trình nhận hai tham số: đường dẫn CSV đầu vào và đường dẫn file CSV đầu ra. Logic chính dùng Spark RDD transformations như `mapPartitions`, `flatMap`, `reduceByKey`, `sortBy`, `saveAsTextFile`; không dùng DataFrame query hoặc Spark SQL cho phần tính toán.

Lệnh chạy mẫu:

```
1 cat > /tmp/task1_1.runner.scala <<'EOF'
2 :load src/Task_1-1/source/Task1_1.scala
3 Task1_1.main(Array("data/input/Amazon Sale Report.csv",
4   "data/output/Task_1-1.csv"))
5 sys.exit(0)
6 EOF
7 SPARK_LOCAL_IP=127.0.0.1 SPARK_LOCAL_HOSTNAME=localhost \
8   spark-shell < /tmp/task1_1.runner.scala
```

Parse và lọc purchase hợp lệ.

```
1 val purchases = parsedRows.flatMap { case (epochDay, state, size, status,
2   qtyText) =>
3   parsePositiveInt(qtyText).filter(_ => status.contains("shipped")).flatMap {
4     qty =>
5     if (state.nonEmpty && size.nonEmpty) Some(Purchase(epochDay, state, size,
6     qty))
7     else None
8   }
9 }
```

Sinh đóng góp cho sliding window.

```
1 val windowContributions = purchases.flatMap { purchase =>
2   (1L to 7L).iterator.flatMap { offset =>
3     val targetDay = purchase.orderEpochDay + offset
4     if (targetDay >= minEpochDay) {
5       Some(((purchase.state, targetDay, purchase.size), purchase.qty))
6     } else {
7       None
8     }
9   }
10 }
```

Cộng tổng quantity và chọn size thắng.

```
1 val sizeTotals = windowContributions.reduceByKey(_ + _)
2
3 val winners = sizeTotals
```

```

4  .map { case ((state, day, size), totalQty) => ((state, day), (size, totalQty))
    }
5  .reduceByKey { (left, right) =>
6      val (leftSize, leftQty) = left
7      val (rightSize, rightQty) = right
8      if (leftQty > rightQty) left
9      else if (rightQty > leftQty) right
10     else if (leftSize <= rightSize) left
11     else right
12 }

```

3.5 Phân tích chuyên sâu

3.5.1 Pseudocode tổng quan

Algorithm 1 Sliding Window — Most Bought Size per State (MapReduce)

Require: Dataset $\mathcal{D}$ with columns (Date, Status, Qty, Size, ship-state)

Ensure: CSV file: (State, Date d , MostBoughtSize, MaxQty)

1: $d_{\min} \leftarrow \text{min epoch day in } \mathcal{D}$

— MAP PHASE —

2: **for all** row $r \in \mathcal{D}$ where Status contains “shipped” $\wedge$ Qty > 0 **do**

3: $t \leftarrow \text{epochDay}(r.\text{Date})$

4: **for** $\delta = 1$ **to** 7 **do**

5: **if** $d_{\min} \leq t + \delta$ **then**

6: **emit** $\langle (r.\text{State}, t + \delta, r.\text{Size}), r.\text{Qty} \rangle$

7: **end if**

8: **end for**

9: **end for**

— REDUCE PHASE 1 —

10: **for all** key (State, Day, Size) **do**

11: $\text{totalQty} \leftarrow \sum \text{values}$

▷ `reduceByKey(_ + _)`

12: **end for**

— REDUCE PHASE 2 —

13: **for all** key (State, Day) **do**

14: $\text{winner} \leftarrow \arg \max_{\text{Size}} \text{totalQty}$

▷ tie-break: lexicographic

15: **emit** (State, Day, winner.Size, winner.Qty)

16: **end for**

Phạm vi của sổ trượt cho ngày tính toán d được định nghĩa:

$$W(d) = [d - 7, d - 1]$$

Tương đương, một giao dịch tại ngày t đóng góp cho các ngày tính toán $d \in [t + 1, t + 7]$.

3.5.2 Tại sao vòng lặp lồng nhau ngày thơ bất khả thi

Cách tiếp cận trực tiếp là: với mỗi ngày tính toán d và mỗi bang s , quét toàn bộ bộ dữ liệu để lọc các dòng có ngày thuộc khoảng $[d - 7, d - 1]$ và cùng bang, rồi đếm số lượng theo kích cỡ. Gọi D là số ngày khác nhau, S là số bang, N là tổng số dòng dữ liệu; phương pháp này có chi phí thời gian lên tới $O(D \times S \times N)$. Với $D \approx 120$, $S \approx 36$, $N \approx 128\,000$, số phép so sánh cần thực hiện là $\sim 5.5 \times 10^8$ — quá tốn kém và hoàn toàn không tận dụng được kiến trúc tính toán song song của mô hình MapReduce.

3.5.3 Thiết kế khóa giải quyết vấn đề thế nào

Thay vì quét ngược từ ngày tính toán d về quá khứ, lời giải áp dụng cơ chế “đẩy xuôi” từ phía giao dịch: một đơn hàng xảy ra tại ngày t sẽ tự động phát sinh ra tối đa 7 bản ghi đóng góp cho các ngày tính toán tương lai từ $t + 1$ đến $t + 7$. Khóa được phát ra là $(\text{state}, \text{targetDay}, \text{size})$, chuyển đổi cơ chế của sổ trượt thành bài toán gom nhóm khóa - giá trị thông thường:

- **Bước giảm lần 1 (reduceByKey):** Cộng tổng số lượng cho mỗi khóa $(\text{state}, \text{day}, \text{size})$. Chi phí: mỗi giao dịch phát sinh tối đa 7 đóng góp ($O(7N)$ bản ghi trung gian), sau đó tiến hành phân tán và cộng dồn song song.
- **Bước giảm lần 2:** Đổi khóa thành $(\text{state}, \text{day})$ để gom toàn bộ các kích cỡ cùng bang và ngày tính toán, từ đó chọn ra kích cỡ có tổng số lượng lớn nhất. Nếu xảy ra hòa, phá hòa bằng thứ tự từ điển tăng dần của chuỗi kích cỡ.

Phương pháp này chỉ tốn chi phí $O(7N)$ cho bước ánh xạ và $O(K \log K)$ cho bước sắp xếp và giảm (với K là số lượng khóa khác nhau), hiệu quả hơn nhiều bậc so với cách tiếp cận ngây thơ.

3.5.4 Xử lý các ngày trống không có giao dịch

Ví dụ: nếu không có đơn hàng nào xảy ra vào ngày 04-05-22 nhưng có các đơn hàng trong khoảng từ 03-29-22 đến 04-04-22, ngày 04-05-22 vẫn phải xuất hiện trong kết quả đầu ra vì cửa sổ [03 – 29, 04 – 04] của nó có dữ liệu đóng góp.

Cơ chế phát đóng góp tự động giải quyết vấn đề này: giao dịch mua tại ngày t khi phát đóng góp cho $t + 1$ đến $t + 7$ sẽ tự động điền dữ liệu vào các ngày trống này. Đồng thời, đối với các đơn hàng cuối cùng (ví dụ ngày 06-29-22), cửa sổ trượt cho phép kết quả kéo dài đến ngày 07-06-22 do các giao dịch cuối này vẫn nằm trong cửa sổ quá khứ của những ngày tiếp sau.

3.6 Kết quả thực thi & Xuất dữ liệu

Tệp kết quả đầu ra đã được tạo tại `data/output/Task_1-1.csv`. Tệp có 3,544 dòng bao gồm dòng tiêu đề, tương ứng 3,543 dòng kết quả. Một số dòng đầu của kết quả:

```
1 ship-state,date,most_bought_size,max_quantity
2 ANDAMAN & NICOBAR,04-02-22,M,1
3 ANDAMAN & NICOBAR,04-03-22,S,2
4 ANDAMAN & NICOBAR,04-04-22,S,3
5 ANDAMAN & NICOBAR,04-05-22,S,3
6 ANDAMAN & NICOBAR,04-06-22,S,3
7 ANDAMAN & NICOBAR,04-07-22,S,4
8 ANDAMAN & NICOBAR,04-08-22,S,4
9 ANDAMAN & NICOBAR,04-09-22,XL,5
```

Kiểm chứng thủ công. Với bang ANDAMAN & NICOBAR và ngày tính toán 04-09-22, cửa sổ quá khứ được xác định từ ngày 04-02-22 đến hết ngày 04-08-22. Sau khi lọc các đơn hàng hợp lệ (Status chứa `shipped` và số lượng `Qty > 0`), tổng số lượng theo kích cỡ trong cửa sổ quá khứ này là:

Size	Tổng Qty trong window
XL	5
L	4
S	4
XS	4
XXL	3
3XL	1
M	1

Bảng 6: Thống kê tổng số lượng (Qty) theo kích cỡ trong cửa sổ quá khứ tại bang Andaman & Nicobar vào ngày 04-09-22

Kích cỡ XL có tổng số lượng cao nhất là 5, khớp hoàn toàn với dòng kết quả đầu ra **ANDAMAN & NICOBAR,04-09-22,XL,5**. Phép kiểm chứng này xác nhận cơ chế phát đóng góp hoạt động hoàn toàn chính xác.

3.7 Minh chứng bằng hình ảnh

Các hình dưới đây là minh chứng chạy chương trình, xem trước kết quả đầu ra và kiểm chứng thủ công cho Task 1.1.

```
scala> Task1_1.main(Array("data/input/Amazon Sale Report.csv", "data/output/Task_1-1.csv"))
26/05/19 15:28:06 WARN SparkSession: Using an existing Spark session; only runtime SQL configurations will take effect.
Task 1-1 completed. Output written to data/output/Task_1-1.csv

scala> sys.exit(0)
```

Hình 1: Kết quả chạy chương trình Task 1.1

1	ship-state	date	most_bought_size	max_quantity
2	ANDAMAN & NI	04-02-22	M	1
3	ANDAMAN & NI	04-03-22	S	2
4	ANDAMAN & NI	04-04-22	S	3
5	ANDAMAN & NI	04-05-22	S	3
6	ANDAMAN & NI	04-06-22	S	3
7	ANDAMAN & NI	04-07-22	S	4
8	ANDAMAN & NI	04-08-22	S	4
9	ANDAMAN & NI	04-09-22	XL	5
10	ANDAMAN & NI	04-10-22	XS	5
11	ANDAMAN & NI	04-11-22	XL	4
12	ANDAMAN & NI	04-12-22	XL	4

Hình 2: Một phần nội dung file Task_1-1.csv

```

May 19 6:40 PM • en 100%
pearspringmind@master:~/Studying/Big Data/Lab03/Lab03_BigData

[pearspringmind@master: Lab03_BigData]$ python check_1.1.py
=====
KIỂM CHỨNG THỦ CÔNG TASK 1.1: ANDAMAN & NICOBAR
Cửa sổ trượt (7 ngày trước: ngày 04-09-22): 02-04-22 -> 08-04-22
=====

[1] Danh sách các đơn hàng hợp lệ trong cửa sổ:
-----
date      | size | qty | status
-----
04-02-22  | S    | 1   | Shipped
04-02-22  | S    | 1   | Shipped
04-02-22  | XL   | 1   | Shipped
04-03-22  | L    | 1   | Shipped
04-03-22  | L    | 1   | Shipped
04-03-22  | S    | 1   | Shipped
04-03-22  | XS   | 1   | Shipped
04-03-22  | XS   | 1   | Shipped
04-03-22  | XXL  | 1   | Shipped
04-04-22  | XS   | 1   | Shipped
04-05-22  | XXL  | 1   | Shipped - Delivered to Buyer
04-06-22  | S    | 1   | Shipped
04-06-22  | XL   | 1   | Shipped - Picked up
04-06-22  | XL   | 1   | Shipped - Picked up
04-06-22  | XXL  | 1   | Shipped - Picked up
04-07-22  | M    | 1   | Shipped
04-07-22  | XL   | 1   | Shipped
04-08-22  | L    | 1   | Shipped - Delivered to Buyer
04-08-22  | L    | 1   | Shipped - Delivered to Buyer
04-08-22  | XL   | 1   | Shipped
04-08-22  | XS   | 1   | Shipped
04-08-22  | XXL  | 1   | Shipped

[2] Tổng hợp số lượng (qty) theo từng Size:
-----
size      | Tổng qty
-----
XL        | 5
L         | 4
S         | 4
XS        | 4
XXL       | 3
3XL       | 1
M         | 1

[3] Kết luận phần tử thắng cuộc (Most Bought Size):
-> Size bán chạy nhất: XL
-> Tổng số lượng (Max qty): 5
-> Kết quả nhập với output: ANDAMAN & NICOBAR,04-09-22,XL,5
=====
[pearspringmind@master: Lab03_BigData]$

```

Hình 3: Kiểm chứng thủ công sliding window cho một state-date mẫu

4 Task 1.2: State-level Median Variety per Month

4.1 Phân tích truy vấn

Bài toán yêu cầu tính giá trị trung vị của độ đa dạng sản phẩm cấp bang cho mỗi tháng. Cụ thể, với mỗi cặp (`ship-state`, `month`), ta cần xác định giá trị trung vị của “variety” trên tập hợp các kiểu dáng thỏa mãn điều kiện đã phục vụ ít nhất một kích cỡ từ XXL trở lên.

Định nghĩa độ đa dạng. Độ đa dạng của một kiểu dáng trong phạm vi (`state`, `month`) là số lượng mã sản phẩm phân biệt (SKU) gắn với kiểu dáng đó trong cùng phạm vi. Độ đa dạng không phải là số lượng đơn hàng hay số lượng kích cỡ, mà là số mã sản phẩm duy nhất đã xuất hiện cho kiểu dáng đó tại bang và tháng đang xét.

Điều kiện kiểu dáng hợp lệ. Một kiểu dáng được xem là hợp lệ nếu trong cùng phạm vi (`state`, `month`, `style`) tồn tại ít nhất một bản ghi có cột kích cỡ `Size` thuộc nhóm từ XXL trở lên. Các ký hiệu kích cỡ được hỗ trợ bao gồm: XXL, 2XL, XXXL, 3XL, 4XL, 5XL, 6XL. Ngoài ra, hàm kiểm tra trong mã nguồn còn sử dụng biểu thức chính quy $([2-9]|\{2,\})XL$ để bao phủ các kích cỡ lớn hơn nữa (ví dụ 7XL, 10XL).

Điểm quan trọng. Sau khi một kiểu dáng được xác định là hợp lệ, độ đa dạng của kiểu dáng đó vẫn phải đếm tất cả các mã sản phẩm phân biệt (SKU) trong phạm vi (`state`, `month`), kể cả các mã sản phẩm có kích cỡ nhỏ hơn XXL. Điều kiện kích cỡ $\geq XXL$ chỉ dùng để quyết định kiểu dáng có được đưa vào tập tính trung vị hay không, hoàn toàn không dùng để lọc bỏ mã sản phẩm.

Cách tính trung vị. Sau khi thu được danh sách độ đa dạng của tất cả kiểu dáng hợp lệ trong một cặp (`state`, `month`), danh sách này được sắp xếp tăng dần. Nếu số lượng phần tử lẻ, giá trị trung vị là phần tử ở vị trí chính giữa; nếu chẵn, trung vị là trung bình cộng của hai phần tử ở giữa. Kết quả được làm tròn đến đúng 4 chữ số phần thập phân.

Cấu trúc dữ liệu đầu ra. Tập kết quả `Task_1-2.csv` gồm có 3 cột:

Cột	Kiểu dữ liệu	Ý nghĩa
<code>ship-state</code>	String	Bang/tỉnh đã chuẩn hóa viết hoa
<code>month</code>	String	Tháng, định dạng YYYY-MM
<code>median_variety</code>	Double (4 d.p.)	Giá trị trung vị của các kiểu dáng hợp lệ

Bảng 7: Định nghĩa schema tập đầu ra `Task_1-2.csv`

4.2 Phân rã thành các bước cơ bản

Lời giải được triển khai theo tư duy MapReduce trên nền tảng Spark RDD, gồm các bước sau:

1. **Đọc dữ liệu đầu vào và chọn cột.** Chương trình đọc tệp CSV bằng `sc.textFile` trên đường dẫn tuyệt đối, phân tích dòng đầu bằng bộ phân tích CSV tự viết và kiểm tra đủ 5 cột cần thiết: `Date`, `SKU`, `Style`, `Size`, `ship-state`. Các dòng thiếu một trong 5 trường này bị bỏ qua. Cột `ship-state` and `Size` được loại bỏ khoảng trắng và chuẩn hóa viết hoa. Cột `Date` được phân tích cú pháp từ các định dạng ngày sang định dạng tháng `YYYY-MM`; các dòng có ngày không hợp lệ bị bỏ qua.
2. **Ánh xạ sang khóa kiểu dáng.** Mỗi bản ghi hợp lệ được phát ra dưới dạng cặp khóa - giá trị với khóa là `(state, month, style)` và giá trị là `(SKU, isAtLeastXXL)`. Đây là bước ánh xạ đầu tiên, phát tán dữ liệu theo khóa phức hợp 3 thành phần để chuẩn bị cho bước gom nhóm tiếp theo.
3. **Bước giảm lần 1 – tính độ đa dạng và cờ XXL.** Sử dụng phép biến đổi `aggregateByKey` với bộ tích lũy là một tập hợp động `mutable.Set[String]` (chứa các mã sản phẩm phân biệt) và một cờ nhị phân `hasXXL`. Hàm tích lũy từng dòng (`seqOp`) thêm mã sản phẩm vào tập hợp và áp dụng phép toán logic OR cho giá trị `isAtLeastXXL` vào cờ. Hàm kết hợp các phân vùng (`combOp`) hợp nhất hai tập hợp và OR hai cờ lại với nhau. Kết quả: mỗi khóa `(state, month, style)` cho ra cặp giá trị `(variety = |set|, hasXXL)`.
4. **Lọc kiểu dáng hợp lệ.** Chỉ giữ lại các kiểu dáng có `hasXXL = true`. Bước lọc này được thực hiện trước khi đổi khóa để giảm lượng dữ liệu cần phân tán qua mạng ở bước tiếp theo.
5. **Ánh xạ sang khóa bang - tháng.** Đổi khóa từ `(state, month, style)` thành khóa mới `(state, month)`, giá trị tương ứng là `variety` (số nguyên). Thành phần kiểu dáng trong khóa bị loại bỏ vì từ bước này trở đi chương trình chỉ cần gom nhóm độ đa dạng theo bang và tháng.
6. **Bước giảm lần 2 – tính trung vị.** Sử dụng phép biến đổi `groupByKey` để gom toàn bộ độ đa dạng của các kiểu dáng hợp lệ cùng cặp `(state, month)` thành một danh sách. Danh sách này được sắp xếp tăng dần rồi chuyển qua hàm `computeMedian` để tính giá trị trung vị, kết quả cuối cùng được định dạng thành chuỗi có 4 chữ số phần thập phân.

7. **Xuất tệp CSV.** Kết quả được sắp xếp theo bang và tháng (`sortBy`) rồi thu thập (`collect()`) về nút điều phối. Chương trình sử dụng thư viện `PrintWriter` của Java I/O để ghi trực tiếp dòng tiêu đề và từng dòng kết quả ra một tệp CSV duy nhất, tránh việc sinh ra thư mục chứa nhiều tệp bộ phận đặc trưng của Hadoop.

4.3 Lý do phân rã

Bài toán đòi hỏi hai tầng gom nhóm vì hai lý do chính. Thứ nhất, cùng một kiểu dáng có thể chứa cả mã sản phẩm kích cỡ nhỏ (S, M, L) lẫn mã sản phẩm kích cỡ lớn (XXL, 3XL). Nếu lọc chỉ giữ các dòng có kích cỡ $\geq$ XXL trước khi đếm độ đa dạng, ta sẽ mất các mã sản phẩm kích cỡ nhỏ và sai định nghĩa độ đa dạng. Do đó, bộ giảm đầu tiên phải đồng thời duy trì toàn bộ tập hợp các mã sản phẩm duy nhất và ghi nhận kiểu dáng đã từng phục vụ kích cỡ từ XXL trở lên.

Thứ hai, bài toán yêu cầu tính trung vị – một phép toán toàn cục trên danh sách đã sắp xếp, không thể thực hiện bằng một phép reduce đơn giản kiểu cộng dồn. Vì vậy cần tầng gom nhóm thứ hai: gom tất cả độ đa dạng theo khóa (`state`, `month`), sắp xếp danh sách rồi trích giá trị trung vị. Cách phân rã thành hai tầng reducer hoàn toàn phù hợp với mô hình xử lý khóa - giá trị vì mỗi bộ giảm chỉ cần xử lý dữ liệu cùng khóa mà không phụ thuộc vào các khóa khác.

4.4 Chiến lược thực thi & Triển khai mã nguồn

Task 1.2 được cài đặt trong file `src/Task_1-2/source/Task1_2.scala`. Chương trình nhận hai tham số: đường dẫn CSV đầu vào và đường dẫn file CSV đầu ra. Toàn bộ phần xử lý chính dùng Spark RDD/key-value transformations, gồm `textFile`, `flatMap`, `aggregateByKey`, `filter`, `map`, `groupByKey`, `sortBy`; không dùng DataFrame query hoặc Spark SQL cho logic bài toán MapReduce.

Lệnh chạy mẫu:

```
1 cat > /tmp/task1_2.runner.scala <<'EOF'
2 :load src/Task_1-2/source/Task1_2.scala
3 Task1_2.main(Array("data/input/Amazon Sale Report.csv",
4   "data/output/Task1_2.csv"))
5 sys.exit(0)
6 EOF
7 spark-shell < /tmp/task1_2.runner.scala
```

Xác định size threshold.

```

1 def isAtLeastXXL(size: String): Boolean = {
2   val s = size.trim.toUpperCase
3   val xlSizes = Set("XXL", "2XL", "XXXL", "3XL",
4                     "4XL", "5XL", "6XL")
5   xlSizes.contains(s) || s.matches("[2-9]\\d{2,}XL")
6 }
    
```

Hàm `isAtLeastXXL` trước tiên kiểm tra kích cỡ có thuộc tập các giá trị phổ biến hay không. Nếu không, biểu thức chính quy sẽ bắt các kích cỡ dạng `nXL` với $n \geq 2$ hoặc số có từ hai chữ số trở lên, đảm bảo bao phủ mọi kích cỡ từ XXL trở lên mà không nhận lầm các giá trị như XL hay 1XL.

Map và Reduce lần 1 – tính variety per style.

```

1 val mapped = dataLines.flatMap(line => {
2   val fields = parseCsvLine(line)
3   val dateStr = field(fields, col.date).trim
4   val sku     = field(fields, col.sku).trim
5   val style   = field(fields, col.style).trim
6   val size    = field(fields, col.size).trim.toUpperCase
7   val state   = field(fields, col.state).trim.toUpperCase
8   if (dateStr.isEmpty || sku.isEmpty || style.isEmpty ||
9       size.isEmpty || state.isEmpty) None
10  else parseMonth(dateStr).map(month =>
11    ((state, month, style), (sku, isAtLeastXXL(size))))
12 })
13
14 val styleVariety = mapped.aggregateByKey((Set.empty[String], false))(
15   (acc, value) => {
16     (acc._1 + value._1, acc._2 || value._2)
17   },
18   (acc1, acc2) => {
19     (acc1._1 ++ acc2._1, acc1._2 || acc2._2)
20   }
21 ).map {
22   case ((state, month, _), (skuSet, hasXXL)) =>
23     ((state, month), (skuSet.size, hasXXL))
24 }
    
```

Reduce lần 2 – lọc qualifying styles và tính median.

```
1 val medianVariety = styleVariety
2   .filter { case (_, (_, hasXXL)) => hasXXL }
3   .map { case (k, (size, _)) => (k, size) }
4   .groupByKey()
5   .flatMap { case ((state, month), sizesIter) =>
6     val varieties = sizesIter.toList.sorted
7     computeMedian(varieties).map(median =>
8       (state, month, median))
9   }
```

Hàm tính median.

```
1 def computeMedian(sortedValues: List[Int]):
2   Option[String] = {
3   if (sortedValues.isEmpty) None
4   else {
5     val n = sortedValues.size
6     val median =
7       if (n % 2 == 1) sortedValues(n / 2).toDouble
8       else (sortedValues(n / 2 - 1) +
9         sortedValues(n / 2)) / 2.0
10    Some(f"$median%.4f")
11  }
12 }
```

Xử lý dữ liệu lỗi. Các dòng có giá trị null ở bất kỳ cột nào trong 5 cột cần thiết đều bị loại bỏ ngay từ đầu quy trình xử lý. Hàm `parseMonth` trả về `None` khi chuỗi ngày không đúng định dạng ngày tháng, và `flatMap` tự động bỏ qua các giá trị `None` này. Cột `ship-state` và `Size` được `trim` và chuyển `toUpperCase` để chuẩn hóa, tránh trường hợp cùng một bang nhưng bị viết khác kiểu chữ (ví dụ “Bihar” và “BIHAR”) dẫn đến trùng lặp khóa.

4.5 Phân tích chuyên sâu

4.5.1 Pseudocode tổng quan

Algorithm 2 State-level Median Variety per Month (Two-pass MapReduce)

Require: Dataset $\mathcal{D}$ with columns (Date, SKU, Style, Size, ship-state)

Ensure: CSV file: (State, Month, MedianVariety)

— **MAP** —

```

1: for all row  $r \in \mathcal{D}$  where all 5 columns  $\neq$  null do
2:   month  $\leftarrow$  parseMonth(r.Date)                                ▷ “YYYY-MM”
3:   emit  $\langle (r.State, month, r.Style), (r.SKU, isXXL(r.Size)) \rangle$ 
4: end for

```

— **REDUCE 1: variety per style** —

```

5: for all key (State, Month, Style) do
6:   skuSet  $\leftarrow$  collect distinct SKU from values
7:   hasXXL  $\leftarrow \bigvee$  isXXL flags from values
8:   variety  $\leftarrow |\text{skuSet}|$ 
9:   if hasXXL then
10:    emit  $\langle (State, Month), variety \rangle$ 
11:   end if
12: end for

```

— **REDUCE 2: median** —

```

13: for all key (State, Month) do
14:    $V \leftarrow$  sort(all variety values)
15:    $n \leftarrow |V|$ 
16:   if  $n$  is odd then
17:     median  $\leftarrow V[\lfloor n/2 \rfloor]$ 
18:   else
19:     median  $\leftarrow (V[n/2 - 1] + V[n/2]) / 2$ 
20:   end if
21:   emit (State, Month, median)
22: end for

```

Công thức tính trung vị cho danh sách đã sắp xếp $V = [v_1, v_2, \dots, v_n]$:

$$\text{median}(V) = \begin{cases} v_{\lceil n/2 \rceil} & \text{nếu } n \text{ lẻ} \\ \frac{v_{n/2} + v_{n/2+1}}{2} & \text{nếu } n \text{ chẵn} \end{cases}$$

4.5.2 Tại sao trung vị KHÔNG thể tính bằng Combiner đơn giản

Trong mô hình MapReduce, Combiner cho phép giảm bớt dữ liệu trước bước phân tán bằng cách thực hiện tính toán giảm cục bộ tại mỗi tiến trình ánh xạ. Tuy nhiên, Combiner chỉ hoạt động hiệu quả với các phép toán mang tính chất kết hợp và giao hoán như sum, min, max. Trung vị **không** thỏa mãn các tính chất này: trung vị của hai tập hợp con không bằng trung vị của tập hợp gộp.

Ví dụ: Phân vùng A có độ đa dạng [1, 3, 5], giá trị trung vị = 3. Phân vùng B có độ đa dạng [2, 4], trung vị = 3. Nhưng tập hợp gộp [1, 2, 3, 4, 5] có trung vị = 3 — mặc dù trùng hợp trong ví dụ này nhưng không được đảm bảo trong trường hợp tổng quát (ví dụ nếu A = [1, 10], B = [2, 3]: trung vị A = 5.5, trung vị B = 2.5, nhưng trung vị gộp = 2.5).

Do đó, bước Reduce 2 bắt buộc phải sử dụng `groupByKey` (không phải `reduceByKey`) để gom toàn bộ danh sách độ đa dạng về cùng một bộ giảm, tiến hành sắp xếp rồi trích vị trí giữa. Đây là chi phí hiệu năng bắt buộc để truyền tải toàn bộ các giá trị độ đa dạng qua mạng.

4.5.3 Chiến lược tính trung vị trong mô hình MapReduce

Lời giải áp dụng chiến lược **quy trình gom nhóm hai bước**:

1. **Bước 1** (`aggregateByKey`): Gom nhóm theo (`state`, `month`, `style`) để tính độ đa dạng (độ đa dạng bằng $|\text{distinct SKU}|$) và cờ đánh dấu `hasXXL`. `aggregateByKey` cho phép sử dụng tập hợp động `mutable.Set` làm bộ tích lũy — hiệu quả hơn `groupByKey` vì chỉ cần duy trì tập hợp duy nhất thay vì gom toàn bộ danh sách.
2. **Bước 2** (`groupByKey`): Gom nhóm theo (`state`, `month`) để thu thập tất cả các giá trị độ đa dạng, sắp xếp rồi tính trung vị. Ở bước này, `groupByKey` là bắt buộc vì phép tính trung vị cần toàn bộ danh sách đã được sắp xếp.

Lý do không sử dụng `TreeMap/SortedMap` trong bộ giảm: trong ngữ cảnh Spark RDD, phép `aggregateByKey` với `SortedMap` vẫn phải thực hiện trộn các cấu trúc map từ các phân vùng khác nhau — chi phí trộn $O(k \log k)$ tương đương với sắp xếp sau khi chạy `groupByKey`, nhưng lại làm phức tạp hóa mã nguồn và tăng mức tiêu thụ bộ nhớ do phải duy trì cấu trúc cây trong bộ tích lũy.

4.5.4 Cách xác định “kích cỡ $\geq$ XXL” — logic phân tích

Hàm `isAtLeastXXL` kiểm tra kích cỡ của sản phẩm theo hai bước:

1. **So khớp trực tiếp:** So khớp trực tiếp với tập hợp định sẵn {XXL, 2XL, XXXL, 3XL, 4XL, 5XL, 6XL} sau khi loại bỏ khoảng trắng và chuyển chữ hoa.
2. **Biểu thức chính quy:** Biểu thức $([2-9]|\d{2,})XL$ bắt các kích cỡ dạng nXL với $n \geq 2$ hoặc n có từ 2 chữ số trở lên (ví dụ 10XL, 12XL).

Thứ tự kiểm tra này rất quan trọng: kiểm tra trong tập hợp trước (chi phí $O(1)$) vì đa số các trường hợp dữ liệu thực tế rơi vào đây; biểu thức chính quy chỉ đóng vai trò dự phòng cho các kích cỡ hiếm gặp hơn. Lưu ý kích cỡ XL (chỉ có 1 chữ X) và 1XL đều **không** hợp lệ vì không khớp với tập hợp định sẵn và biểu thức chính quy yêu cầu hệ số $n \geq 2$.

4.6 Kết quả thực thi & Xuất dữ liệu

Tập kết quả đầu ra đã được tạo tại `data/output/Task_1-2.csv`. Thông tin chi tiết:

- Đường dẫn kết quả: `data/output/Task_1-2.csv`.
- Số dòng bao gồm dòng tiêu đề: 129.
- Số dòng kết quả thực tế: 128.
- Kích thước tệp: 3,270 bytes.

Một số dòng đầu của kết quả đầu ra:

```
1 ship-state,month,median_variety
2 ANDAMAN & NICOBAR,2022-04,1.0000
3 ANDAMAN & NICOBAR,2022-05,1.0000
4 ANDAMAN & NICOBAR,2022-06,1.0000
5 ANDHRA PRADESH,2022-03,1.0000
6 ANDHRA PRADESH,2022-04,2.0000
7 ANDHRA PRADESH,2022-05,2.0000
8 ANDHRA PRADESH,2022-06,2.0000
9 ARUNACHAL PRADESH,2022-04,1.0000
10 ASSAM,2022-04,2.0000
```

Kiểm chứng thủ công. Chọn trường hợp bang CHANDIGARH và tháng 2022-03 làm mẫu kiểm tra. Sau khi lọc trực tiếp trên file dữ liệu gốc `Amazon Sale Report.csv`, ta thu được đúng 1 bản ghi:

Date	Style	SKU	Size
03-31-22	JNE3758	JNE3758-KR-XXXL	3XL

Bảng 8: Dữ liệu đơn hàng gốc tại bang Chandigarh vào tháng 03-2022 để kiểm chứng

Phân tích: chỉ có 1 kiểu dáng (JNE3758) với 1 mã sản phẩm phân biệt, do đó độ đa dạng variety = 1. Kiểu dáng này có kích cỡ $3XL \geq XXL$ nên hợp lệ. Danh sách độ đa dạng hợp lệ: [1]. Giá trị trung vị của tập [1] bằng 1.0000. Kết quả này khớp hoàn toàn với dòng tương ứng trong tệp kết quả đầu ra: CHANDIGARH,2022-03,1.0000, xác nhận logic MapReduce hoạt động chính xác.

4.7 Minh chứng bằng hình ảnh

Các hình dưới đây là minh chứng chạy chương trình, xem trước kết quả đầu ra và kiểm chứng thủ công cho Task 1.2.

```
scala> Task1_2.main(Array("data/input/Amazon Sale Report.csv", "data/output/Task_1-2.csv"))
26/05/13 17:42:05 WARN SparkSession: Using an existing Spark session; only runtime SQL configurations will take effect.
Task 1-2 hoàn thành: 128 dòng -> data/output/Task_1-2.csv
```

Hình 4: Kết quả chạy chương trình Task 1.2

ship-state	month	median_variety
ANDAMAN & NICOBAR	2022-04	1
ANDAMAN & NICOBAR	2022-05	1
ANDAMAN & NICOBAR	2022-06	1
ANDHRA PRADESH	2022-03	1
ANDHRA PRADESH	2022-04	2
ANDHRA PRADESH	2022-05	2
ANDHRA PRADESH	2022-06	2
ARUNACHAL PRADESH	2022-04	1
ARUNACHAL PRADESH	2022-05	1
ARUNACHAL PRADESH	2022-06	1
ASSAM	2022-04	2
ASSAM	2022-05	1.5
ASSAM	2022-06	1
BIHAR	2022-04	2
BIHAR	2022-05	2
BIHAR	2022-06	1
CHANDIGARH	2022-03	1
CHANDIGARH	2022-04	1

Hình 5: Một phần nội dung file Task_1-2.csv

```

=====
KIỂM CHỨNG THỦ CÔNG: CHANDIGARH, Tháng 03-2022
=====

[1] Bảng dữ liệu gốc sau khi lọc (Filter):
      Date      Style      SKU Size
03-31-22 JNE3758 JNE3758-KR-XXXL 3XL

[2] Phân tích từng Style:
-> Style: JNE3758
+ Danh sách SKUs (1 SKUs): ['JNE3758-KR-XXXL']
+ Danh sách Sizes: ['3XL']
=> HỢP LỆ (Có size >= XXL)

[3] Kết quả cuối cùng:
-> Tập các variety hợp lệ để tính Median: [1]
-> MEDIAN VARIETY tính bằng tay: 1.0000
=====

```

Hình 6: Kiểm chứng thủ công median variety cho một state-month mẫu

5 Task 2.1: Percentage of Cancelled Orders per City

5.1 Phân tích truy vấn

Bài toán yêu cầu tính **phần trăm đơn hàng bị hủy** theo từng thành phố (`ship-city`), trong đó chỉ xét các đơn hàng thỏa mãn **đồng thời ba điều kiện** sau:

Điều kiện 1 — Mức dịch vụ. Đơn hàng phải có mức dịch vụ giao hàng `ship-service-level` bằng "Standard". Trong mã nguồn, phép so sánh được chuẩn hóa bằng `lower(trim(col("ship-service-level"))` === "standard" để tránh sai lệch do khoảng trắng thừa hoặc khác biệt giữa chữ hoa và chữ thường.

Điều kiện 2 — Có ít nhất 3 khuyến mãi hợp lệ về mặt thời gian. Mỗi đơn hàng chứa cột `promotion-ids`, là chuỗi các mã khuyến mãi được phân tách bằng dấu phẩy. Một chương trình khuyến mãi được gọi là *hợp lệ về mặt thời gian* nếu khoảng thời gian hoạt động của nó trên **toàn bộ tập dữ liệu** kéo dài ít nhất 2 ngày. Cụ thể, với mỗi mã khuyến mãi duy nhất xuất hiện trong bộ dữ liệu:

- Xác định ngày xuất hiện **đầu tiên** ($d_{\min}$) và ngày xuất hiện **cuối cùng** ($d_{\max}$).

- Tính khoảng thời gian hoạt động: $\text{active_days} = d_{\max} - d_{\min}$ (tính bằng số ngày).
- Khuyến mãi hợp lệ khi và chỉ khi $\text{active_days} \geq 2$.

Một đơn hàng thỏa mãn điều kiện này nếu trong danh sách `promotion-ids` của nó chứa **ít nhất 3** mã khuyến mãi hợp lệ, kể cả các khuyến mãi do Amazon phát hành.

Điều kiện 3 — Giá trị đơn hàng (Amount) thấp hơn mức trung bình của bang. Tính giá trị trung bình cột `Amount` theo từng bang `ship-state` trên tập con các đơn hàng thỏa mãn `Fulfilment = "Merchant"` và `Courier Status = "Shipped"`. Đơn hàng đang xét chỉ được giữ lại nếu:

$$\text{Amount}_{\text{order}} < \overline{\text{Amount}}_{\text{Merchant, Shipped}}^{(\text{cùng bang})}$$

Tính phần trăm đơn hàng bị hủy. Sau khi lọc ra tất cả đơn hàng thỏa mãn đồng thời cả ba điều kiện trên, chương trình tiến hành gom nhóm theo thành phố `ship-city` đã được chuẩn hóa để tính toán:

$$\text{cancelled_percentage} = \frac{|\{o : \text{Status}(o) = \text{"Cancelled"}\}|}{|\text{tổng số đơn hàng hợp lệ}|} \times 100$$

Mã nguồn xác định trạng thái đơn hàng bị hủy bằng phép so khớp `lower(trim(col("Status")))` `=== "cancelled"` để đảm bảo tính chính xác sau khi chuẩn hóa.

Cấu trúc dữ liệu đầu ra:

Cột	Kiểu	Ý nghĩa
<code>ship-city</code>	String	Thành phố giao hàng đã chuẩn hóa viết hoa
<code>cancelled_percentage</code>	Double	Tỷ lệ hủy (%), làm tròn 4 chữ số thập phân

Bảng 9: Định nghĩa schema tập đầu ra `Task_2-1.parquet`

5.2 Phân rã thành các bước cơ bản

Lời giải được phân rã thành 9 bước xử lý tuần tự, mỗi bước tương ứng với một phép biến đổi DataFrame rõ ràng:

1. **Đọc dữ liệu CSV và chuyển kiểu dữ liệu (df).** Đọc tệp CSV với dòng đầu chứa tiêu đề, tắt tính năng tự động suy luận kiểu dữ liệu (`inferSchema`) để kiểm soát kiểu dữ liệu

một cách chủ động. Phân tích cột `Date` sang kiểu ngày (`date`) theo định dạng `MM-dd-yy`, chuyển kiểu cột `Amount` sang `DoubleType`, chuẩn hóa tên thành phố `ship-city` và bang `ship-state` bằng hàm `upper(trim(...))`, đồng thời tạo cột định danh `order_row_id` bằng hàm `monotonically_increasing_id()` để định danh duy nhất cho từng đơn hàng sau các phép liên kết. `DataFrame` này được lưu tạm vào bộ nhớ (`cache()`) vì sẽ được tái sử dụng trong nhiều nhánh xử lý khác nhau.

2. **Tách các mã khuyến mãi thành từng dòng riêng** (`promoExploded`). Lọc bỏ các dòng có cột `promotion-ids` rỗng hoặc mang giá trị null. Sử dụng hàm `split` để tách chuỗi khuyến mãi theo dấu phẩy, dùng hàm `transform` kết hợp với `trim` để loại bỏ khoảng trắng dư thừa của từng mã, sau đó dùng hàm `explode` để phân rã danh sách thành từng dòng mã khuyến mãi riêng lẻ. Bước này rất quan trọng vì khoảng thời gian hoạt động là thuộc tính mang tính chất toàn cục của từng mã khuyến mãi — cần thu thập toàn bộ các ngày xuất hiện của mã trên toàn bộ tập dữ liệu.
3. **Tính toán thời gian hợp lệ của khuyến mãi** (`promoValidity`). Gom nhóm theo mã khuyến mãi `promo_id`, tính ngày xuất hiện sớm nhất `min(parsed_date)` và muộn nhất `max(parsed_date)`, sau đó sử dụng hàm hiệu ngày `datediff` để tính số ngày hoạt động `active_days`. Chỉ giữ lại các mã khuyến mãi có số ngày hoạt động `active_days` ≥ 2 .
4. **Đếm số khuyến mãi hợp lệ theo từng đơn hàng bằng DataFrame**. Từ `DataFrame` gốc `df`, chương trình thực hiện phân rã các mã khuyến mãi theo định danh đơn hàng `order_row_id`, liên kết với bảng khuyến mãi hợp lệ `promoValidity`, sau đó gom nhóm theo `order_row_id` để đếm số lượng khuyến mãi hợp lệ duy nhất bằng hàm `countDistinct`. Cách làm này giữ nguyên tính chất declarative của Structured API, không sử dụng hàm tự định nghĩa (UDF) và không phải thu thập dữ liệu về nút điều phối, giúp Catalyst dễ dàng tối ưu hóa.
5. **Áp dụng Điều kiện 1 và 2** (`afterCond12`). Lọc các đơn hàng có `ship-service-level` = "Standard", sau đó thực hiện phép liên kết trái với bảng đếm số khuyến mãi hợp lệ `validPromotionCounts` theo `order_row_id`; các đơn hàng không có khuyến mãi nào hợp lệ sẽ nhận giá trị mặc định là 0 thông qua hàm `coalesce`. Cuối cùng, thực hiện lọc lấy các đơn hàng có `valid_promo_count` ≥ 3 .

6. **Tính toán giá trị trung bình Amount theo bang (stateAvgDf).** Từ DataFrame gốc df (không phải afterCond12), lọc các bản ghi có Fulfilment = "Merchant" và Courier Status = "Shipped", loại bỏ các dòng có giá trị Amount bị null, sau đó thực hiện gom nhóm và tính trung bình: `groupBy("ship_state_norm").agg(avg("Amount"))`. Kết quả thu được là một bảng dữ liệu nhỏ chứa ngưỡng chỉ tiêu trung bình cho từng bang.
7. **Liên kết và áp dụng Điều kiện 3 (qualifiedDf).** Thực hiện phép liên kết trong giữa afterCond12 và stateAvgDf trên cột bang ship_state_norm, sau đó lọc lấy các đơn hàng có Amount < merchant_shipped_avg. Sau bước này, ta thu được tập hợp các đơn hàng thỏa mãn đồng thời cả ba điều kiện lọc.
8. **Tính phần trăm đơn hàng bị hủy theo từng thành phố (resultDf).** Gom nhóm theo thành phố ship_city_norm, đếm tổng số đơn hàng bằng `count(*)` và đếm số đơn hàng bị hủy bằng hàm tổng điều kiện `sum(when(... === "cancelled", 1).otherwise(0))`. Tính tỷ lệ phần trăm đơn hàng bị hủy và làm tròn đến 4 chữ số thập phân.
9. **Xuất tập Parquet duy nhất.** Sử dụng `coalesce(1)` để gom dữ liệu về một phân vùng duy nhất, ghi vào một thư mục tạm trên ổ đĩa, sau đó đổi tên tập bộ phận thành tên tập Parquet đơn lẻ chính xác theo đường dẫn đầu ra và dọn dẹp các tập phụ checksum.

5.3 Lý do phân rã

Tính khoảng hoạt động hợp lệ của khuyến mãi trên toàn bộ tập dữ liệu trước. Khoảng thời gian hoạt động của một mã khuyến mãi được xác định bởi khoảng cách giữa ngày xuất hiện đầu tiên và ngày xuất hiện cuối cùng của nó trên toàn bộ tập dữ liệu, chứ không thể chỉ xét riêng lẻ trong từng đơn hàng. Do đó, quy trình phân rã-gom nhóm-hiệu ngày bắt buộc phải được xử lý trên toàn bộ dữ liệu trước khi áp dụng ngược lại cho từng đơn hàng. Đây là lý do quy trình xử lý được chia thành hai nhánh song song: một nhánh tính toán độ hợp lệ của các mã khuyến mãi và nhánh chính xử lý dữ liệu đơn hàng.

Ngưỡng chỉ tiêu trung bình theo bang phải được tính riêng rồi liên kết sau. Điều kiện thứ ba yêu cầu so sánh giá trị Amount của mỗi đơn hàng với mức trung bình của bang tương ứng nhưng chỉ trên tập hợp con các đơn hàng của Merchant và đã Shipped. Vì tập hợp con này có tính chất khác biệt hoàn toàn với tập hợp đơn hàng Standard đang xét, chúng ta không thể sử dụng

trực tiếp hàm của sổ mà bắt buộc phải tính toán trung bình riêng biệt thành bảng `stateAvgDf` rồi thực hiện phép liên kết theo cột `ship_state_norm`. Vì bảng `stateAvgDf` có kích thước rất nhỏ, Spark có thể tự động áp dụng phép liên kết phát tán `BroadcastHashJoin` để tránh chi phí phân tán dữ liệu qua mạng.

Thứ tự áp dụng các điều kiện lọc giúp tối ưu hóa chi phí shuffle. Mã nguồn chủ động áp dụng Điều kiện 1 (lọc các đơn Standard) trước khi liên kết với bảng đếm số khuyến mãi để giảm thiểu số lượng đơn hàng tham gia vào phép liên kết. Điều kiện 3 (so sánh giá trị trung bình của bang) được áp dụng sau cùng vì nó phụ thuộc vào kết quả liên kết với bảng `stateAvgDf`. Nếu đảo ngược thứ tự — liên kết với bảng trung bình của bang trước rồi mới lọc mức dịch vụ và khuyến mãi — khối lượng dữ liệu tham gia vào phép liên kết và chi phí shuffle trung gian sẽ tăng lên rất nhiều.

5.4 Chiến lược thực thi & Triển khai mã nguồn

Task 2.1 được cài đặt trong file `src/Task_2-1/source/Task2_1.scala`, sử dụng hoàn toàn `DataFrame/Dataset` API — tuyệt đối không dùng các chuỗi truy vấn Spark SQL dạng thô. Chương trình nhận hai tham số: đường dẫn CSV đầu vào và đường dẫn file Parquet đầu ra.

Lệnh chạy mẫu:

```
1 cat > /tmp/task2_1.runner.scala <<'EOF'
2 :load src/Task_2-1/source/Task2_1.scala
3 Task2_1.main(Array("data/input/Amazon Sale Report.csv",
4   "data/output/Task_2-1.parquet"))
5 sys.exit(0)
6 EOF
7 spark-shell < /tmp/task2_1.runner.scala
```

Explode và tính promotion validity.

```
1 val promoExploded = df
2   .filter(col("promotion-ids").isNotNull
3     && col("promotion-ids") != "")
4   .select(col("parsed_date"),
5     explode(transform(
6       split(col("promotion-ids"), ","),
7       (promoId: Column) => trim(promoId)
```

```

8    )).as("promo_id"))
9    .filter(col("promo_id") != "")
10
11 val promoValidity = promoExploded
12    .groupBy("promo_id")
13    .agg(min("parsed_date").as("first_date"),
14         max("parsed_date").as("last_date"))
15    .withColumn("active_days",
16               datediff(col("last_date"), col("first_date")))
17    .filter(col("active_days") >= 2)
18    .select("promo_id")

```

Lý do chương trình sử dụng hàm biến đổi `transform` kết hợp với `trim` trước khi chạy `explode` thay vì cắt khoảng trắng sau khi phân rã là: đảm bảo mỗi mã khuyến mãi được làm sạch ngay trong mảng trước khi phân rã dòng mới, tránh sinh ra các dòng trống vô nghĩa do các dấu phẩy dư thừa ở cuối chuỗi.

Đếm valid promotions theo order bằng DataFrame.

```

1 val orderPromos = df
2   .filter(col("promotion-ids").isNotNull &&
3          trim(col("promotion-ids")) != "")
4   .select(col("order_row_id"),
5           explode(transform(split(col("promotion-ids"), ","),
6                                (promoId: Column) => trim(promoId))).as("promo_id"))
7   .filter(col("promo_id") != "")
8
9 val validPromotionCounts = orderPromos
10  .join(broadcast(promoValidity), Seq("promo_id"), "inner")
11  .groupBy("order_row_id")
12  .agg(countDistinct("promo_id").as("valid_promo_count"))

```

Chiến lược này giữ trọn vẹn tính chất declarative của Structured API. Bảng `promoValidity` chỉ chứa 185 khuyến mãi hợp lệ nên được Spark tự động phát tán làm nhánh dựng bằng **Build side** khi liên kết với `orderPromos`; phần đếm số lượng theo khóa `order_row_id` là phép gom nhóm rõ ràng, hiển thị trực quan trong kế hoạch vật lý.

Filter service level và đếm promotions.

```

1 val afterCond12 = df

```

```

2 .filter(lower(trim(
3   col("ship-service-level"))) === "standard")
4 .join(validPromotionCounts, Seq("order_row_id"), "left")
5 .withColumn("valid_promo_count",
6   coalesce(col("valid_promo_count"), lit(0L)))
7 .filter(col("valid_promo_count") >= 3)

```

Tính trung bình Amount theo state.

```

1 val stateAvgDf = df
2 .filter(
3   lower(trim(col("Fulfilment"))) === "merchant" &&
4   lower(trim(col("Courier Status"))) === "shipped")
5 .filter(col("Amount").isNotNull)
6 .groupBy("ship_state_norm")
7 .agg(avg("Amount").as("merchant_shipped_avg"))

```

Join điều kiện 3 và tính cancelled percentage.

```

1 val joinedDf = afterCond12.join(
2   stateAvgDf, Seq("ship_state_norm"), "inner")
3
4 val qualifiedDf = joinedDf
5 .filter(col("Amount").isNotNull)
6 .filter(col("Amount") < col("merchant_shipped_avg"))
7
8 val resultDf = qualifiedDf
9 .groupBy(col("ship_city_norm"))
10 .agg(
11   count("*").as("total_orders"),
12   sum(when(lower(trim(col("Status"))) === "cancelled",
13     1).otherwise(0)).as("cancelled_orders"))
14 .withColumn("cancelled_percentage",
15   round(col("cancelled_orders") /
16     col("total_orders") * 100.0, 4))
17 .select(col("ship_city_norm").as("ship-city"),
18   col("cancelled_percentage"))
19 .orderBy("ship-city")

```

Việc chuẩn hóa tên thành phố ship-city trước khi gom nhóm là cực kỳ cần thiết để loại bỏ các

sai lệch do chữ hoa/thường và khoảng trắng thừa. Kết quả cuối cùng bám sát yêu cầu của đề bài: tính tỷ lệ phần trăm cho từng thành phố.

5.5 Phân tích chuyên sâu — Kế hoạch thực thi

5.5.1 Đầu ra của lệnh `explain(true)`

Dưới đây là kế hoạch thực thi chi tiết thu được từ lệnh `resultDf.explain(true)` khi chạy chương trình:

```

1 == Physical Plan ==
2 AdaptiveSparkPlan isFinalPlan=false
3 +- Sort [ship-city ASC NULLS FIRST], true, 0
4   +- Exchange rangepartitioning(ship-city ASC NULLS FIRST, 200)
5     +- HashAggregate(keys=[ship_city_norm], functions=[sum(...), count(1)])
6       +- Exchange hashpartitioning(ship_city_norm, 200)
7         +- HashAggregate(keys=[ship_city_norm], functions=[partial_sum(...), partial_count(1)])
8           +- Project [Status, ship_city_norm]
9             +- BroadcastHashJoin [ship_state_norm], [ship_state_norm], Inner, BuildRight,
10              (Amount < merchant_shipped_avg), false
11              :- Project [Status, Amount, ship_city_norm, ship_state_norm]
12                :- +- BroadcastHashJoin [order_row_id], [order_row_id], Inner, BuildRight, false
13                  :- Filter lower(trim(ship-service-level)) = standard
14                  :- +- BroadcastExchange HashedRelationBroadcastMode(order_row_id)
15                    :- +- Filter coalesce(valid_promo_count, 0) >= 3
16                    :- +- HashAggregate(keys=[order_row_id],
17                      functions=[count(distinct promo_id)])
18                    :- +- Exchange hashpartitioning(order_row_id, 200)
19                    :- +- HashAggregate(keys=[order_row_id, promo_id])
20                    :- +- Exchange hashpartitioning(order_row_id, promo_id, 200)
21                    +- BroadcastHashJoin [promo_id], [promo_id],
22                     Inner, BuildRight, false
23                     :- Generate explode(transform(split(promotion-ids, ","), trim))
24                     +- BroadcastExchange HashedRelationBroadcastMode(promo_id)
25                     +- HashAggregate(keys=[promo_id],
26                       functions=[min(parsed_date), max(parsed_date)])
27                     +- Exchange hashpartitioning(promo_id, 200)
28 +- BroadcastExchange HashedRelationBroadcastMode(ship_state_norm)
29   +- HashAggregate(keys=[ship_state_norm], functions=[avg(Amount)])
30     +- Exchange hashpartitioning(ship_state_norm, 200)

```

5.5.2 Phân tích chiến lược liên kết vật lý

Quy trình xử lý chứa một phép liên kết chính: `afterCond12` $\bowtie_{\text{ship_state_norm}}$ `stateAvgDf`.

Join	Strategy thực tế	Giải thích
<code>orderPromos</code> <code>promoValidity</code>	⊗ BroadcastHashJoin	<code>promoValidity</code> chỉ có 185 dòng nên được broadcast làm build side.
<code>standardOrders</code> <code>validPromotionCounts</code>	⊗ BroadcastHashJoin	Bảng <code>order</code> đủ điều kiện promotion sau aggregation nhỏ hơn ngưỡng broadcast, Spark chọn BuildRight.
<code>afterCond12</code> <code>stateAvgDf</code>	⊗ BroadcastHashJoin	<code>stateAvgDf</code> chỉ có bảng trung bình theo state, nhỏ hơn nhiều so với broadcast threshold mặc định 10 MB.

Bảng 10: Chi tiết các phép liên kết trong kế hoạch thực thi vật lý

Nhánh dựng bảng **Build side** của cả ba phép liên kết phát tán BroadcastHashJoin đều nằm ở phía bên phải trong kế hoạch vật lý: `promoValidity`, `validPromotionCounts`, và `stateAvgDf`. Spark tiến hành tuần tự hóa các bảng nhỏ này, phát tán đến các nút thực thi, sau đó thực hiện tìm kiếm bảng băm cục bộ trên phân vùng của bảng lớn hơn. Nhờ đó, các phép liên kết chính không cần phải thực hiện phân tán lại dữ liệu cả hai phía theo khóa liên kết.

Nếu `stateAvgDf` vượt quá 10 MB (cấu hình mặc định `spark.sql.autoBroadcastJoinThreshold`), Spark sẽ chuyển sang phép liên kết SortMergeJoin — yêu cầu shuffle cả hai bảng theo khóa liên kết — tốn kém hiệu năng hơn rất nhiều. Trong trường hợp cụ thể này, bảng chỉ có ~36 dòng nên BroadcastHashJoin là sự lựa chọn tối ưu nhất.

5.5.3 Đếm các nút trao đổi dữ liệu

Dựa trên kế hoạch vật lý thực tế, các nút trao đổi dữ liệu phân tán chính gồm có:

#	Exchange	Mục đích
1	hashpartitioning(promo_id)	Gom ngày xuất hiện của từng promotion để tính min/max date.
2	hashpartitioning(order_row_id, promo_id)	Khử trùng lặp cặp order-promotion trước khi countDistinct.
3	hashpartitioning(order_row_id)	Gom các promotion hợp lệ theo từng order.
4	hashpartitioning(ship_state_norm)	Tính trung bình Amount cho Merchant+Shipped theo state.
5	hashpartitioning(ship_city_norm)	Gom các đơn qualified theo city để tính percentage.
6	rangepartitioning(ship-city)	Sort toàn cục output cuối.

Bảng 11: Phân tích các nút trao đổi dữ liệu và mục đích sử dụng

Lưu ý: Kế hoạch thực thi còn có các nút phát tán **BroadcastExchange** dành cho các bảng **promoValidity**, **validPromotionCounts** và **stateAvgDf**; đây là cơ chế phát tán bảng nhỏ, hoàn toàn khác với các phép shuffle phân tán dữ liệu của các bảng lớn.

5.5.4 Đếm các giai đoạn thực thi

Mỗi nút trao đổi dữ liệu shuffle Exchange sẽ tạo ra một ranh giới phân tách giai đoạn thực thi mới. Với 6 nút shuffle Exchange chính và 3 nút BroadcastExchange, quy trình xử lý kết quả cần khoảng **7–9 giai đoạn thực thi** cho hành động ghi kết quả đầu ra; số stage thực tế có thể thay đổi nhẹ tùy theo các hành động phụ như `count()` hay `show()` phục vụ việc ghi nhật ký.

1. **Stage 1:** Đọc tệp CSV, phân tích ngày tháng và giá trị đơn hàng, duyệt toàn bộ dữ liệu đầu vào.
2. **Stage 2:** Phân rã mã khuyến mãi, shuffle theo **promo_id**, tính toán ngày bắt đầu và kết thúc.
3. **Stage 3:** Lọc đơn hàng Merchant+Shipped, shuffle theo **ship_state_norm**, tính Amount trung bình.
4. **Stage 4:** Phân rã khuyến mãi đơn hàng, liên kết phát tán với **promoValidity**, shuffle theo **order_row_id**.
5. **Stage 5:** Lọc mức dịch vụ Standard, thực hiện phép liên kết BroadcastHashJoin với **validPromotionCounts**.

6. **Stage 6:** Thực hiện phép liên kết BroadcastHashJoin với `stateAvgDf`, lọc giá trị đơn hàng hợp lệ.
7. **Stage 7:** Shuffle dữ liệu theo `ship_city_norm`, tính tổng số đơn và đơn bị hủy.
8. **Stage 8:** Sắp xếp toàn cục, gộp phân vùng và ghi dữ liệu ra định dạng Parquet.

5.5.5 Thảo luận về cơ chế tự thích ứng khi thực thi truy vấn

Trong mã nguồn của Task 2.1, chương trình sử dụng cấu hình mặc định của Spark runtime cho cơ chế tự thích ứng AQE (`spark.sql.adaptive.enabled`). Với môi trường thực thi Spark 3.4.0, kế hoạch vật lý hiển thị từ khóa `AdaptiveSparkPlan`, xác nhận AQE đang hoạt động và có thể mang lại các tối ưu hóa sau:

- **Tự động gộp phân vùng sau khi shuffle:** gộp các phân vùng nhỏ sau shuffle, giảm số lượng task dư thừa.
- **Tự động chuyển đổi phép liên kết SortMergeJoin sang BroadcastHashJoin:** nếu sau bước lọc dữ liệu, một bên của phép liên kết có kích thước nhỏ hơn ngưỡng quy định, AQE sẽ tự động chuyển đổi chiến lược liên kết để tối ưu hiệu năng.
- **Tối ưu hóa các phép liên kết bị lệch dữ liệu:** tự động chia nhỏ các phân vùng bị lệch dữ liệu lớn thành nhiều phân vùng nhỏ hơn. Trong quy trình này, phép liên kết trên bảng `ship-state` ít bị ảnh hưởng bởi hiện tượng lệch dữ liệu vì mỗi bảng chỉ chứa duy nhất một dòng kết quả trung bình.

Trong kế hoạch thực thi minh họa ở trên, dòng chữ `AdaptiveSparkPlan isFinalPlan=false` xuất hiện do lệnh `resultDf.explain(true)` được gọi trước khi thực thi hành động ghi dữ liệu đầu ra. Tại thời điểm đó, Spark mới chỉ hiển thị kế hoạch thích ứng ban đầu; AQE vẫn cần thu thập phản hồi trong lúc chạy từ các giai đoạn shuffle trước để đưa ra kế hoạch tối ưu cuối cùng sau khi hành động ghi được thực hiện. Vì vậy, trạng thái `isFinalPlan=false` hoàn toàn phản ánh đúng nguyên lý hoạt động của AQE.

5.6 Kết quả thực thi & Xuất dữ liệu

- Đường dẫn đầu ra: `data/output/Task_2-1.parquet`.

- **Định dạng:** Tập Parquet duy nhất, có thể đọc dễ dàng bằng thư viện Pandas (`pd.read_parquet`) hoặc Spark ở chế độ chạy cục bộ.
- **Số dòng kết quả:** 2,452 nhóm thành phố.
- **Kích thước tệp:** phụ thuộc vào siêu dữ liệu Parquet của lần ghi dữ liệu thực tế.

Bảng tổng hợp trích xuất từ tệp Parquet đầu ra:

Chỉ số	Giá trị
Số dòng kết quả đầu ra	2,452
Số thành phố duy nhất	2,452
Giá trị nhỏ nhất của <code>cancelled_percentage</code>	0.0
Giá trị lớn nhất của <code>cancelled_percentage</code>	0.0
Số thành phố có tỷ lệ đơn bị hủy <code>cancelled_percentage > 0</code>	0

Bảng 12: Các chỉ số thống kê tổng quan trích xuất từ tệp Parquet đầu ra của Task 2.1

Một số dòng kết quả mẫu:

1	ship-city	cancelled_percentage
2	(CHIKMAGALUR DISTRICT).	(N.R PUR THALUKU) 0.0
3	7BARASAT	0.0
4	ABHAYAPURI	0.0
5	ABOHAR	0.0
6	ABU ROAD	0.0
7	ACHALPUR	0.0
8	ACHANTA	0.0
9	ADAJAN GAM, SURAT	0.0
10	ADALAJ	0.0
11	ADBHAR	0.0

Kiểm chứng thủ công. Chọn thành phố mẫu `ship-city = BENGALURU`. Sau khi áp dụng đủ ba điều kiện lọc của đề bài, truy vấn kiểm chứng thủ công thu được 1,318 đơn hàng hợp lệ, trong đó không có đơn hàng nào mang trạng thái bị hủy (`cancelled_orders = 0`). Các dòng đơn hàng mẫu đều thỏa mãn các mức dịch vụ Standard, số khuyến mãi hợp lệ ≥ 3 và giá trị đơn hàng nhỏ hơn mức trung bình của bang tương ứng. Do đó:

$$\text{cancelled_percentage} = \frac{0}{1318} \times 100 = 0.0000$$

Giá trị này khớp hoàn toàn với kết quả đầu ra BENGALURU, 0.0. Toàn bộ tỷ lệ hủy trong lần chạy dữ liệu hiện tại đều bằng 0 vì không có đơn hàng hợp lệ nào mang trạng thái **Cancelled**.

5.7 Minh chứng bằng hình ảnh

Các hình ảnh dưới đây cung cấp minh chứng trực quan về việc chạy chương trình, kế hoạch thực thi Catalyst plan và kết quả kiểm tra tệp dữ liệu đầu ra của Task 2.1.

```
=====
TASK 2.1 RUN SUMMARY
=====
[Task 2-1] Loaded 128975 rows from data/input/Amazon Sale Report.csv
[Task 2-1] Found 185 temporally-valid promotions
[Task 2-1] After Cond 1+2: 32303 orders
[Task 2-1] After all 3 conditions: 17047 qualifying orders
[Task 2-1] Result preview:
+-----+-----+
|ship-city|cancelled_percentage|
+-----+-----+
|(CHIKMAGALUR DISTERICT). (N.R PUR THALUKU)|0.0|
|7BARASAT|0.0|
|ABHAYAPURI|0.0|
|ABOHAR|0.0|
|ABU ROAD|0.0|
+-----+-----+
[Task 2-1] Total cities in result: 2452
[Task 2-1] Output written to data/output/Task_2-1.parquet
[Task 2-1] Completed successfully.
```

Hình 7: Kết quả chạy chương trình Task 2.1

BÁO CÁO LAB 3

```
May 19 6:56 PM en 100%
pearspringmind@master:~/Studying/Big Data/Lab03/Lab03_BigData

[Task 2-1] Total cities in result: 2947
=====
EXECUTION PLAN - explain(true)
=====
== Parsed Logical Plan ==
'sort ['ship-state ASC NULLS FIRST, 'ship-city ASC NULLS FIRST], true
+- Project [ship-city#33, ship-state#3090, total_orders#4555L, cancelled_orders#4557L, round(((cast(cancelled_orders#4557L as double) / cast(total_orders#4555L as double)) * 100.0), 4) AS cancelled_percentage#4562]
  +- Aggregate [ship-city#33, ship-state#3090], [ship-city#33, ship-state#3090, count(1) AS total_orders#4555L, sum(CASE WHEN (lower(trim(Status#20, None)) = cancelled) THEN 1 ELSE 0 END) AS cancelled_orders#4557L]
    +- Filter (Amount#92 < merchant_shipped_avg#2403)
      +- Filter isnotnull(Amount#92)
        +- Project [index#17, Order ID#18, Date#19, Status#20, Fulfillment#21, Sales Channel #22, ship-service-level#23, Style#24, SKU#25, Category#26, Size#27, ASIN#28, Courier Status#29, Qty#30, currency#31, Amount#92, ship-city#33, ship-postal-code#35, ship-country#36, promotion-ids#37, B2B#38, fulfilled-by#39, Unnamed: 22#40, parsed_date#65, ... 4 more fields]
          +- Join Inner, (ship-state#34 = ship-state#3090)
            +- Filter (valid_promo_count#1603 >= 3)
              +- Project [index#17, Order ID#18, Date#19, Status#20, Fulfillment#21, Sales Channel #22, ship-service-level#23, Style#24, SKU#25, Category#26, Size#27, ASIN#28, Courier Status#29, Qty#30, currency#31, Amount#92, ship-city#33, ship-postal-code#35, ship-country#36, promotion-ids#37, B2B#38, fulfilled-by#39, Unnamed: 22#40, ... 3 more fields]
                +- Project [index#17, Order ID#18, Date#19, Status#20, Fulfillment#21, Sales Channel #22, ship-service-level#23, Style#24, SKU#25, Category#26, Size#27, ASIN#28, Courier Status#29, Qty#30, currency#31, cast(Amount#32 as double) AS Amount#92, ship-city#33, ship-state#34, ship-postal-code#35, ship-country#36, promotion-ids#37, B2B#38, fulfilled-by#39, Unnamed: 22#40, parsed_date#65]
                  +- Project [index#17, Order ID#18, Date#19, Status#20, Fulfillment#21, Sales Channel #22, ship-service-level#23, Style#24, SKU#25, Category#26, Size#27, ASIN#28, Courier Status#29, Qty#30, currency#31, Amount#32, ship-city#33, ship-state#34, ship-postal-code#35, ship-country#36, promotion-ids#37, B2B#38, fulfilled-by#39, Unnamed: 22#40, to_date(Date#19, Some('MM-dd-yy'), Some(Asia/Ho_Chi_Minh)) AS parsed_date#65]
                    +- Relation [index#17,Order ID#18,Date#19,Status#20,Fulfillment#21,Sales Channel #22,ship-service-level#23,Style#24,SKU#25,Category#26,Size#27,ASIN#28,Courier Status#29,Qty#30,currency#31,Amount#32,ship-city#33,ship-postal-code#35,ship-country#36,promotion-ids#37,B2B#38,fulfilled-by#39,Unnamed: 22#40] csv
                      +- Aggregate [ship-state#3090], [ship-state#3090, avg(Amount#92) AS merchant_shipped_avg#2403]
                        +- Filter isnotnull(Amount#92)
                          +- Filter ((lower(trim(Fulfillment#3077, None)) = merchant) AND (lower(trim(Courier Status#3085, None)) = shipped))
                            +- Project [index#3073, Order ID#3074, Date#3075, Status#3076, Fulfillment#3077, Sales Channel #3078, ship-service-level#3079, Style#3080, SKU#3081, Category#3082, Size#3083, ASIN#3084, Courier Status#3085, Qty#3086, currency#3087, Amount#92, ship-city#3089, ship-state#3090, ship-postal-code#3091, ship-country#3092, promotion-ids#3093, B2B#3094, fulfilled-by#3095, Unnamed: 22#3096, ... 2 more fields]
                              +- Project [index#3073, Order ID#3074, Date#3075, Status#3076, Fulfillment#3077, Sales Channel #3078, ship-service-level#3079, Style#3080, SKU#3081, Category#3082, Size#3083, ASIN#3084, Courier Status#3085, Qty#3086, currency#3087, cast(Amount#3088 as double) AS Amount#92, ship-city#3089, ship-state#3090, ship-postal-code#3091, ship-country#3092, promotion-ids#3093, B2B#3094, fulfilled-by#3095, Unnamed: 22#3096, parsed_date#65]
                                +- Project [index#3073, Order ID#3074, Date#3075, Status#3076, Fulfillment#3077, Sales Channel #3078, ship-service-level#3079, Style#3080, SKU#3081, Category#3082, Size#3083, ASIN#3084, Courier Status#3085, Qty#3086, currency#3087, Amount#3088, ship-city#3089, ship-state#3090, ship-postal-code#3091, ship-country#3092, promotion-ids#3093, B2B#3094, fulfilled-by#3095, Unnamed: 22#3096, to_date(Date#3075, Some('MM-dd-yy'), Some(Asia/Ho_Chi_Minh)) AS parsed_date#65]
                                  +- Relation [index#3073,Order ID#3074,Date#3075,Status#3076,Fulfillment#3077,Sales Channel #3078,ship-service-level#3079,Style#3080,SKU#3081,Category#3082,Size#3083,ASIN#3084,Courier Status#3085,Qty#3086,currency#3087,Amount#3088,ship-city#3089,ship-state#3090,ship-postal-code#3091,ship-country#3092,promotion-ids#3093,B2B#3094,fulfilled-by#3095,Unnamed: 22#3096] csv
                                +- Analyzed Logical Plan ==
                                  ship-city: string, ship-state: string, cancelled_percentage: double
                                  Sort [ship-state#3090 ASC NULLS FIRST, ship-city#33 ASC NULLS FIRST], true
                                  +- Project [ship-city#33, ship-state#3090, total_orders#4555L, cancelled_orders#4557L, round(((cast(cancelled_orders#4557L as double) / cast(total_orders#4555L as double)) * 100.0), 4) AS cancelled_percentage#4562]
                                    +- Aggregate [ship-city#33, ship-state#3090], [ship-city#33, ship-state#3090, count(1) AS total_orders#4555L, sum(CASE WHEN (lower(trim(Status#20, None)) = cancelled) THEN 1 ELSE 0 END) AS cancelled_orders#4557L]
                                      +- Filter (Amount#92 < merchant_shipped_avg#2403)
                                        +- Filter isnotnull(Amount#92)
                                          +- Project [index#17, Order ID#18, Date#19, Status#20, Fulfillment#21, Sales Channel #22, ship-service-level#23, Style#24, SKU#25, Category#26, Size#27, ASIN#28, Courier Status#29, Qty#30, currency#31, Amount#92, ship-city#33, ship-postal-code#35, ship-country#36, promotion-ids#37, B2B#38, fulfilled-by#39, Unnamed: 22#40, parsed_date#65, ... 4 more fields]
                                            +- Join Inner, (ship-state#34 = ship-state#3090)
                                              +- Filter (valid_promo_count#1603 >= 3)
                                                +- Project [index#17, Order ID#18, Date#19, Status#20, Fulfillment#21, Sales Channel #22, ship-service-level#23, Style#24, SKU#25, Category#26, Size#27, ASIN#28, Courier Status#29, Qty#30, currency#31, Amount#92, ship-city#33, ship-
```

Hình 8: Kế hoạch thực thi vật lý (Execution plan) của Task 2.1

```
=====
TASK 2.1 PARQUET OUTPUT PREVIEW (Task_2-1.parquet)
=====
+-----+-----+
|ship-city|cancelled_percentage|
+-----+-----+
|(CHIKMAGALUR DISTRICT). (N.R PUR THALUKU)|0.0|
|7BARASAT|0.0|
|ABHAYAPURI|0.0|
|ABOHAR|0.0|
|ABU ROAD|0.0|
|ACHALPUR|0.0|
|ACHANTA|0.0|
|ADAJAN GAM, SURAT|0.0|
|ADALAJ|0.0|
|ADBHAR|0.0|
|ADDANKI|0.0|
|ADDANKI MANDAL BAPATL DISTRICT|0.0|
|ADILABAD|0.0|
|ADIMALY|0.0|
+-----+-----+
only showing top rows

Total records in Task 2.1 output: 2452
Distinct cities: 2452
Min/Max cancelled_percentage: 0.0 / 0.0
Rows with cancelled_percentage > 0: 0
```

Hình 9: Một phần nội dung tệp Parquet kết quả đầu ra Task_2-1.parquet

6 Task 2.2: StdDev of Amount by SKU-Month with Percentile Threshold

6.1 Phân tích truy vấn

Task 2.2 được phụ trách bởi Nguyễn Hồ Anh Tuấn (23120185). Mục tiêu của truy vấn là xét từng nhóm cặp (SKU, month), tìm các đơn hàng có số lượng khuyến mãi đủ lớn dựa theo ngưỡng phân vị động của chính nhóm đó, sau đó tính độ lệch chuẩn tổng thể của giá trị Amount trên tập các đơn hàng được giữ lại. Hai mức phân vị bắt buộc là P90 và P80.

Đơn vị lọc ở đây không phải là giá trị đơn hàng **Amount**, mà là số lượng khuyến mãi của từng đơn hàng. Cột **promotion-ids** được tách ra theo dấu phẩy, loại bỏ khoảng trắng thừa ở mỗi thành phần, bỏ qua các thành phần rỗng và đếm tất cả các mã khuyến mãi còn lại. Các chương trình khuyến mãi do Amazon phát hành vẫn được tính như khuyến mãi thông thường theo yêu cầu đề bài. Nếu cột **promotion-ids** mang giá trị null hoặc rỗng, số lượng khuyến mãi **promo_count** của đơn hàng được đặt mặc định là 0.

Với mỗi nhóm **SKU-month**, chương trình tính hai ngưỡng động: **p90_thresh** và **p80_thresh**. Một đơn hàng được xem là hợp lệ cho mức P90 nếu **promo_count** $\geq$ **p90_thresh**; tương tự, hợp lệ cho P80 nếu **promo_count** $\geq$ **p80_thresh**. Ngưỡng này được tính toán riêng biệt cho từng nhóm cụ thể, hoàn toàn không sử dụng một ngưỡng chung cho toàn bộ tập dữ liệu.

Độ lệch chuẩn tổng thể được tính toán bằng hàm **stddev_pop**, tức là độ lệch chuẩn tổng thể với bậc tự do bằng 0:

$$\sigma = \sqrt{\frac{\sum_{i=1}^N (x_i - \mu)^2}{N}}$$

Nếu sau khi thực hiện lọc theo phân vị mà nhóm đó có ít hơn 2 đơn hàng, kết quả độ lệch chuẩn của nhóm sẽ được đặt mặc định là 0.0. Quy tắc này giúp tránh việc diễn giải sai lệch cho các nhóm chỉ còn lại 0 hoặc 1 đơn hàng hợp lệ sau khi lọc.

Dữ liệu đầu ra được ghi dưới dạng một tệp Parquet duy nhất tại **data/output/Task_2-2.parquet** với cấu trúc như sau:

Cột	Ý nghĩa
SKU	Mã SKU của sản phẩm
month	Tháng theo định dạng yyyy-MM
p90_stddev_approx	Độ lệch chuẩn Amount khi dùng ngưỡng phân vị gần đúng P90
p90_stddev_exact	Độ lệch chuẩn Amount khi dùng ngưỡng phân vị chính xác P90
p80_stddev_approx	Độ lệch chuẩn Amount khi dùng ngưỡng phân vị gần đúng P80
p80_stddev_exact	Độ lệch chuẩn Amount khi dùng ngưỡng phân vị chính xác P80

Bảng 13: Định nghĩa schema tệp đầu ra Task_2-2.parquet

6.2 Phân rã thành các bước cơ bản

Tập mã nguồn của phần này là **src/Task_2-2/source/Task2_2.scala**. Toàn bộ lời giải sử dụng Spark Structured APIs bằng Scala, tuyệt đối không sử dụng các chuỗi truy vấn Spark SQL dạng

thô.

1. **Đọc dữ liệu CSV và chọn các cột cần thiết.** Chương trình đọc tệp dữ liệu `data/input/Amazon Sale Report.csv` có dòng đầu chứa tiêu đề, giữ lại các cột `SKU`, `Date`, `promotion-ids`, `Amount`. Cột `Amount` được chuyển kiểu sang `DoubleType`; các dòng thiếu thông tin `SKU`, thiếu ngày tháng hoặc không thể chuyển đổi giá trị `Amount` sẽ bị loại bỏ để đảm bảo tính hợp lệ cho phép tính độ lệch chuẩn sau này.
2. **Chuẩn hóa định dạng tháng.** Cột `Date` được phân tích cú pháp bằng nhiều định dạng phổ biến như `M/d/yyyy`, `MM-dd-yy`, `M/d/yy`. Sau khi chuyển đổi thành kiểu ngày hợp lệ, chương trình tạo cột tháng `month = date_format(parsed_date, "yyyy-MM")`.
3. **Tính số lượng khuyến mãi của đơn hàng (`promo_count`).** Cột `promotion-ids` được tách theo dấu phẩy, loại bỏ khoảng trắng ở mỗi mã khuyến mãi và lọc bỏ các mã rỗng. Số lượng mã hợp lệ còn lại là `promo_count`; nếu cột mang giá trị null hoặc rỗng thì mặc định đặt bằng 0.
4. **Tạo DataFrame cơ sở.** DataFrame `baseDf` chỉ giữ lại các cột thông tin tối giản gồm `SKU`, `month`, `promo_count`, `amount_val`. DataFrame này được lưu tạm vào bộ nhớ (`cache()`) vì cả hai phương pháp tính phân vị gần đúng và chính xác đều sẽ tái sử dụng lại tập dữ liệu này.
5. **Tính toán ngưỡng phân vị gần đúng (`approxThresholds`).** Gom nhóm theo cặp `SKU`, `month` và tính toán các ngưỡng phân vị P90 và P80 thông qua hàm `percentile_approx(promo_count, p, 10000)`. Ngưỡng này sau đó được liên kết ngược lại với DataFrame cơ sở `baseDf` để tiến hành lọc lấy các đơn hàng thỏa mãn.
6. **Tính toán ngưỡng phân vị chính xác (`exactThresholds`).** Chương trình sử dụng hàm cửa sổ với cấu trúc `partitionBy("SKU", "month").orderBy(promo_count)` để gán số thứ tự dòng `row_number`. Với mỗi nhóm gồm có N đơn hàng, vị trí phân vị chính xác được xác định bằng phương pháp thứ hạng gần nhất: $\lceil p \times N \rceil$. Giá trị số lượng khuyến mãi `promo_count` tại vị trí xếp hạng này chính là ngưỡng phân vị chính xác cần tìm.
7. **Tính toán độ lệch chuẩn cho từng mức phân vị.** Các đơn hàng thỏa mãn điều kiện `promo_count >= threshold` được gom nhóm theo `SKU`, `month`. Hàm hỗ trợ `computeStddev`

đếm số lượng đơn hàng hợp lệ và tính `stddev_pop(amount_val)`; nếu số lượng đơn hàng ít hơn 2 thì trả về kết quả mặc định là 0.0.

8. **Hợp nhất kết quả cuối cùng.** Các kết quả tính độ lệch chuẩn P90/P80 theo hai phương pháp gần đúng và chính xác được liên kết trái (left join) vào danh sách đầy đủ tất cả các nhóm `SKU-month` trích xuất từ `baseDf`. Các giá trị null được thay thế bằng 0.0 và giữ nguyên độ chính xác cao nhất (Double) của hàm `stddev_pop`.

9. **Ghi tệp Parquet duy nhất.** Hàm `writeParquetOutput` sử dụng phép `coalesce(1)` để tạo một tệp bộ phận duy nhất trong thư mục tạm, sau đó đổi tên tệp này thành đúng tên yêu cầu `Task_2-2.parquet`.

6.3 Lý do phân rã

Bước tính số lượng khuyến mãi `promo_count` bắt buộc phải được thực hiện trước khi tính phân vị, vì phép tính phân vị trong bài toán này không được tính trên giá trị đơn hàng `Amount`. Ngưỡng phân vị được tính toán dựa trên số lượng khuyến mãi của mỗi đơn hàng, trong khi cột giá trị `Amount` chỉ được sử dụng ở bước cuối cùng để tính độ lệch chuẩn cho những đơn hàng đã vượt qua các điều kiện lọc.

Ngưỡng phân vị là thông tin mang tính chất đại diện ở cấp độ nhóm `SKU-month`, trong khi quyết định một đơn hàng có được giữ lại hay lọc bỏ lại nằm ở cấp độ từng dòng đơn hàng riêng lẻ. Do đó, sau khi gom nhóm tính toán để tìm ngưỡng, chương trình bắt buộc phải liên kết (join) bảng ngưỡng ngược lại với DataFrame cơ sở `baseDf`. Cách phân rã này giúp giữ ranh giới rõ ràng giữa phép tính gom nhóm cấp độ cao và phép lọc dữ liệu ở cấp độ dòng.

Hai phương pháp tính phân vị (gần đúng và chính xác) được đồng thời triển khai nhằm đáp ứng chính xác yêu cầu so sánh hiệu năng của đề bài. Phương pháp gần đúng tận dụng hàm gom nhóm tích hợp `percentile_approx`, thường cho tốc độ xử lý nhanh hơn vì không cần thực hiện sắp xếp toàn bộ dữ liệu của mỗi nhóm. Phương pháp chính xác sử dụng hàm cửa sổ, yêu cầu sắp xếp và tính toán thứ hạng gần nhất nên tiêu tốn nhiều chi phí hiệu năng hơn, nhưng bù lại nó đảm bảo tìm được giá trị phân vị chính xác tuyệt đối theo đúng công thức lý thuyết $[p \times N]$.

6.4 Chiến lược thực thi & Triển khai mã nguồn

Chương trình được khởi chạy trực tiếp bằng `spark-shell` theo cấu trúc runner của kho lưu trữ:

```
1 :load src/Task_2-2/source/Task2_2.scala
2 Task2_2.main(Array(
3   "data/input/Amazon Sale Report.csv",
4   "data/output/Task_2-2.parquet"
5 ))
```

Đối tượng `SparkSession` trong mã nguồn sử dụng chế độ chạy cục bộ, thiết lập số phân vùng shuffle `spark.sql.shuffle.partitions = 8` và bật cơ chế tự thích ứng `spark.sql.adaptive.enabled = true`. Thiết lập này rất phù hợp cho môi trường chạy thử nghiệm cục bộ với dữ liệu vừa phải, giúp tránh chi phí dư thừa khi tạo quá nhiều tác vụ nhỏ, đồng thời AQE có thể tự động gộp các phân vùng khi cần thiết.

Phần tiền xử lý dữ liệu trong mã nguồn tập trung vào việc khởi tạo `DataFrame` cơ sở `baseDf`:

```
1 val baseDf = withMonthDf
2   .withColumn("promo_count",
3     when(col("promotion-ids").isNull
4       || trim(col("promotion-ids")) === "",
5       lit(0L))
6     .otherwise(
7       size(filter(
8         split(col("promotion-ids"), ","),
9         (token: Column) => trim(token) != lit("")
10      )).cast(LongType)))
11   .select("SKU", "month", "promo_count", "amount_val")
```

Phương pháp gần đúng thực hiện tính toán cả hai ngưỡng P90 và P80 trong cùng một phép gom nhóm duy nhất:

```
1 val approxThresholds = baseDf
2   .groupBy("SKU", "month")
3   .agg(
4     percentile_approx(col("promo_count"),
5       lit(0.9), lit(10000)).alias("p90_thresh"),
6     percentile_approx(col("promo_count"),
7       lit(0.8), lit(10000)).alias("p80_thresh"))
```

Phương pháp chính xác sử dụng hàm cửa sổ để tìm vị trí thứ hạng gần nhất:

```

1 val rankedDf = baseDf
2   .withColumn("rn",
3     row_number().over(winGroupOrd))
4   .withColumn("total_n",
5     count("*").over(winGroup))
6
7 val withPosDf = rankedDf
8   .withColumn("pos_p90",
9     ceil(lit(0.9) * col("total_n")).cast(LongType))
10  .withColumn("pos_p80",
11    ceil(lit(0.8) * col("total_n")).cast(LongType))

```

Kết quả cuối cùng được ghép lại từ tập hợp tất cả các cặp SKU-month để đảm bảo không bỏ sót bất kỳ nhóm hợp lệ nào:

```

1 val resultDf = allGroups
2   .join(p90ApproxDf, Seq("SKU", "month"), "left")
3   .join(p80ApproxDf, Seq("SKU", "month"), "left")
4   .join(p90ExactDf, Seq("SKU", "month"), "left")
5   .join(p80ExactDf, Seq("SKU", "month"), "left")
6   .select("SKU", "month",
7     "p90_stddev_approx", "p90_stddev_exact",
8     "p80_stddev_approx", "p80_stddev_exact")

```

6.5 Phân tích chuyên sâu

6.5.1 Thuật toán bên trong approx_percentile

Hàm percentile_approx (hoặc approx_percentile) trong Spark được triển khai dựa trên thuật toán nổi tiếng **Greenwald-Khanna** (GK). Thuật toán này duy trì một cấu trúc dữ liệu nén đặc biệt, cho phép ước lượng giá trị phân vị tại bất kỳ điểm nào trên dòng dữ liệu với cam kết sai số lý thuyết:

$$\text{rank}_{\text{exact}}(q) - \epsilon \cdot N \leq \text{rank}_{\text{approx}}(q) \leq \text{rank}_{\text{exact}}(q) + \epsilon \cdot N$$

Trong đó $\epsilon = 1/\text{accuracy}$. Với cấu hình độ chính xác mặc định `accuracy = 10000`, sai số tương đối

tối đa được giới hạn ở mức cực nhỏ là $\frac{1}{10000} = 0.01\%$. Thuật toán GK sở hữu những ưu điểm vượt trội sau:

- **Duyệt dữ liệu một lần:** chỉ cần quét qua dữ liệu đúng một lần duy nhất, hoàn toàn không yêu cầu sắp xếp toàn bộ tập dữ liệu.
- **Tiết kiệm bộ nhớ:** chi phí bộ nhớ chỉ ở mức $O(\frac{1}{\epsilon} \log(\epsilon N))$, nhỏ hơn rất nhiều so với chi phí $O(N)$ của việc sắp xếp dữ liệu để tìm phân vị chính xác.
- **Khả năng trộn:** các tóm tắt dữ liệu nén từ nhiều phân vùng khác nhau có thể được trộn lại một cách hiệu quả — cực kỳ thích hợp với kiến trúc phân tán và shuffle của Spark.

6.5.2 Phân vị chính xác: sắp xếp + đánh số dòng

Phương pháp phân vị chính xác sử dụng **phương pháp thứ hạng gần nhất** thông qua hàm cửa sổ:

1. Phân vùng dữ liệu theo khóa (`SKU`, `month`), sắp xếp cột `promo_count` theo thứ tự tăng dần.
2. Đánh số thứ tự dòng `row_number` (bắt đầu từ 1) cho từng đơn hàng trong mỗi phân vùng.
3. Xác định vị trí phân vị mong muốn bằng công thức: $\text{pos} = \lceil p \times N \rceil$ — làm tròn lên để tìm vị trí thứ hạng gần nhất.
4. Lọc lấy dòng có `rn == pos` để trích xuất ngưỡng phân vị chính xác.

Độ phức tạp thuật toán là $O(N \log N)$ cho mỗi nhóm `SKU-month` phục vụ việc sắp xếp dữ liệu trong phân vùng. Tổng chi phí trên toàn bộ tập dữ liệu là $O(\sum_g N_g \log N_g)$ với g đại diện cho các nhóm. Đây là chi phí cao hơn đáng kể so với thuật toán GK có chi phí khấu hao chỉ là $O(N)$, nhưng đảm bảo tìm được giá trị phân vị chính xác tuyệt đối.

Lý lý chọn phương pháp thứ hạng gần nhất thay vì nội suy tuyến tính: Phương pháp thứ hạng gần nhất đảm bảo rằng giá trị phân vị tìm được luôn là một giá trị thực tế đã xuất hiện và quan sát được trong tập dữ liệu (không phải là một giá trị nội suy giả định). Điều này nhất quán hoàn toàn với cơ chế hoạt động của `approx_percentile`, giúp cho việc so sánh hiệu năng và kết quả giữa hai phương pháp trở nên công bằng hơn.

6.5.3 Khi nào phương pháp gần đúng đủ tốt, khi nào bắt buộc dùng chính xác

Phân vị gần đúng phù hợp khi	Phân vị chính xác cần thiết khi
Nhóm dữ liệu lớn ($N > 100$): sai số rank ϵN là rất nhỏ so với khoảng cách giữa các thứ hạng thực tế	Nhóm dữ liệu nhỏ ($N < 10$): sai số ϵN có thể làm trùng sang thứ hạng liền kề, dẫn đến ngưỡng bị lệch
Số khuyến mãi <code>promo_count</code> phân bố đều: chứa nhiều giá trị phân biệt khác nhau, ít có hiện tượng trùng hạng	<code>promo_count</code> bị lệch dữ liệu/trùng hạng nhiều: nhiều đơn hàng có cùng số lượng khuyến mãi, lệch 1 đơn vị ngưỡng có thể làm thay đổi lớn tập hợp đơn hàng hợp lệ
Mục đích khám phá dữ liệu, phân tích nhanh trên tập dữ liệu lớn	Yêu cầu đối chiếu chính xác tuyệt đối, có khả năng tái lập kết quả cao

Bảng 14: So sánh tiêu chí sử dụng giữa phương pháp phân vị gần đúng và phân vị chính xác

Ảnh hưởng trực tiếp của ngưỡng phân vị đến kết quả độ lệch chuẩn `stddev`: nếu ngưỡng phân vị gần đúng khác biệt so với chính xác, tập hợp các đơn hàng thỏa mãn điều kiện lọc sẽ thay đổi, kéo theo giá trị trung bình và phương sai thay đổi. Trong ngữ cảnh số lượng khuyến mãi `promo_count` có nhiều giá trị trùng lặp (rất phổ biến vì đa số đơn hàng chỉ có từ 0 đến 3 khuyến mãi), việc lệch ngưỡng dù chỉ 1 đơn vị cũng có thể loại bỏ hoặc thêm vào một lượng lớn đơn hàng, gây ra sự thay đổi rất lớn cho kết quả độ lệch chuẩn.

6.6 So sánh hai phương pháp Percentile

Kết quả đo hiệu năng được thực hiện 5 lần trên cùng DataFrame cơ sở `baseDf` đã được lưu tạm vào bộ nhớ. Mỗi lần chạy bao gồm toàn bộ quy trình: tính ngưỡng, liên kết ngưỡng với dữ liệu gốc, lọc đơn hàng hợp lệ, tính độ lệch chuẩn tổng thể và ghi nhận kết quả. Tập dữ liệu sau khi lọc các dòng có `Amount` hợp lệ chứa 121,180 đơn hàng và chia thành 16,345 nhóm cặp `SKU-month`.

Phương pháp	5 lần chạy (giây)	Trung bình (s)	Độ lệch chuẩn (s)
Phân vị gần đúng	2.668, 1.183, 1.057, 0.852, 0.840	1.3200	0.6862
Phân vị chính xác	3.781, 1.828, 1.501, 1.358, 1.391	1.9718	0.9197

Bảng 15: Kết quả đo hiệu năng thời gian chạy giữa hai phương pháp tính phân vị

Phương pháp phân vị gần đúng cho tốc độ xử lý nhanh hơn phương pháp chính xác khoảng 33%

trong lần đo benchmark này, do `percentile_approx` là hàm gom nhóm tích hợp tối ưu sẵn của Spark, tránh được chi phí sắp xếp dữ liệu đắt đỏ cho từng phân vùng `SKU-month`. Phương pháp chính xác bắt buộc phải sử dụng hàm cửa sổ sắp xếp theo `promo_count`, phát sinh nhiều chi phí phân tán và sắp xếp hơn.

Về độ chính xác của kết quả, với tập dữ liệu thực tế hiện tại và cấu hình độ chính xác `accuracy = 10000`, ngưỡng phân vị gần đúng trùng khớp hoàn toàn với phương pháp chính xác ở cả hai mức P90 và P80:

Chỉ số đối chiếu	Giá trị sai lệch
Tổng số nhóm SKU-tháng được so sánh	16,345
Số nhóm bị lệch ngưỡng P90 threshold	0
Số nhóm bị lệch ngưỡng P80 threshold	0
Số nhóm bị lệch tập đơn hàng hợp lệ P90	0
Số nhóm bị lệch tập đơn hàng hợp lệ P80	0
Số dòng kết quả bị lệch giá trị <code>p90_stddev</code>	0
Số dòng kết quả bị lệch giá trị <code>p80_stddev</code>	0

Bảng 16: So sánh mức độ sai lệch kết quả giữa phương pháp gần đúng và chính xác

Không xuất hiện bất kỳ cặp `SKU-month` nào trong dữ liệu thực tế tạo ra tập đơn hàng hợp lệ khác nhau giữa hai phương pháp. Nguyên nhân là do số lượng khuyến mãi `promo_count` là biến số nguyên rời rạc có miền giá trị hẹp, quy mô đơn hàng của từng nhóm không quá lớn và tham số `accuracy` của hàm gần đúng được cấu hình ở mức rất cao. Trong các tập dữ liệu có quy mô lớn hơn nữa hoặc phân bố dữ liệu cực kỳ lệch, phương pháp gần đúng có thể sẽ phát sinh sai lệch; khi đó phương pháp chính xác là sự lựa chọn bắt buộc nếu yêu cầu đối chiếu hoặc tái lập kết quả tuyệt đối được ưu tiên hàng đầu.

6.7 Thảo luận về sự phân mảnh dữ liệu và lệch dữ liệu

Sau khi tiến hành lọc sạch các dòng có giá trị cột `Amount` hợp lệ, tập dữ liệu thu được gồm 121,180 đơn hàng được phân chia vào 16,345 nhóm `SKU-month`. Trong dữ liệu thực tế, các nhóm được phân chia rất nhỏ và đều đặn, không có bất kỳ nhóm nào vượt quá quy mô 1,000 đơn hàng. Nhóm có quy mô lớn nhất là sản phẩm có mã `JNE3405-KR-L` vào tháng 2022-04, đạt tổng cộng 381 đơn hàng.

Mã SKU	Tháng	Số lượng đơn hàng
JNE3405-KR-L	2022-04	381
JNE3797-KR-L	2022-06	366
SET268-KR-NP-XL	2022-04	303
JNE3797-KR-M	2022-06	301
SET268-KR-NP-L	2022-04	300

Bảng 17: Thống kê các nhóm SKU-tháng có số lượng đơn hàng lớn nhất

Vì quy mô của nhóm lớn nhất chỉ dừng lại ở mức 381 đơn hàng, tập dữ liệu hoàn toàn không rơi vào trạng thái lệch dữ liệu nghiêm trọng để phải áp dụng các chiến lược phân vùng lại thủ công phức tạp theo từng khóa liên kết. Việc thiết lập thủ công `repartition("SKU", "month")` có thể mang lại hiệu quả cho các tập dữ liệu khổng lồ có hiện tượng lệch khóa cực mạnh, nhưng với quy mô dữ liệu thử nghiệm cục bộ hiện tại, chi phí shuffle bổ sung từ việc phân vùng lại sẽ lớn hơn nhiều so với lợi ích tối ưu hóa mang lại.

Spark mặc định thiết lập kích thước phân vùng tệp dữ liệu đầu vào khoảng 128 MB cho mỗi phân đoạn đọc. Quy mô của nhóm lớn nhất trong bài toán này nhỏ hơn rất nhiều so với ngưỡng cần thiết để phải phân tách phân vùng riêng biệt. Do đó, cấu hình thực tế `spark.sql.shuffle.partitions = 8` kết hợp cùng cơ chế tự thích ứng AQE là cực kỳ hợp lý: đảm bảo khả năng tính toán song song tối đa trên local mode, không sinh ra quá nhiều tác vụ nhỏ gây thất nút cổ chai hiệu năng, đồng thời tạo điều kiện cho Spark tự động gộp các phân vùng khi cần thiết.

6.8 Kết quả thực thi & Xuất dữ liệu

Chương trình đã xuất thành công tệp kết quả Parquet tại đường dẫn `data/output/Task_2-2.parquet`. Tệp dữ liệu này có thể được đọc lại dễ dàng bằng thư viện Pandas với cấu trúc chuẩn gồm 16,345 dòng dữ liệu và 6 cột thông tin đúng theo schema yêu cầu. Các giá trị độ lệch chuẩn `stddev` được lưu trữ dưới định dạng số thực `Double` độ chính xác cao nhất, hoàn toàn không làm tròn trước khi ghi xuống đĩa.

```

1 Rows: 16345
2 Columns: SKU, month,
3   p90_stddev_approx, p90_stddev_exact,
4   p80_stddev_approx, p80_stddev_exact

```

Một số dòng dữ liệu mẫu có độ lệch chuẩn khác 0 trích xuất từ tệp kết quả đầu ra:

SKU	month	p90_approx	p90_exact	p80_approx	p80_exact
AN209-BIEGE-XL	2022-04	6.0	6.0	6.0	6.0
BL003-50BLACK-B	2022-04	0.0	0.0	3.5	3.5
BL009-61BLACK-B	2022-05	0.0	0.0	2.5	2.5
BL013-62BLACK	2022-05	1.7496355	1.7496355	1.7496355	1.7496355
BL021-71BLACK	2022-05	0.0	0.0	115.5290969	115.5290969

Kiểm chứng thủ công. Chọn cặp thông tin bang AN209-BIEGE-XL và tháng 2022-04: sau khi lọc sạch các dòng có giá trị cột **Amount** hợp lệ, nhóm này chỉ chứa đúng 2 đơn hàng. Cả hai đơn hàng đều có số lượng khuyến mãi **promo_count** = 1; danh sách giá trị Amount tương ứng là [229.0, 241.0]. Với quy mô nhóm $N = 2$, vị trí xếp hạng gần nhất của cả hai phân vị P80 và P90 đều bằng $\lceil p \times 2 \rceil = 2$, ngưỡng lọc tìm được là 1. Cả hai đơn hàng đều thỏa mãn điều kiện lọc để giữ lại, giá trị đơn hàng trung bình là 235.0 và độ lệch chuẩn tổng thể tính được:

$$\sigma = \sqrt{\frac{(229 - 235)^2 + (241 - 235)^2}{2}} = \sqrt{\frac{36 + 36}{2}} = 6.0$$

Tập kết quả đầu ra ghi nhận chính xác giá trị 6.0 cho cả hai phân vị P90/P80 ở cả hai phương pháp tính gần đúng và chính xác, trùng khớp hoàn toàn với kết quả kiểm chứng thủ công ở trên.

6.9 Minh chứng bằng hình ảnh

Các hình ảnh dưới đây cung cấp minh chứng trực quan về kết quả chạy thành công chương trình, kết quả đo hiệu năng thực tế và xem trước nội dung tệp kết quả Parquet đầu ra của Task 2.2.

```
[Task2-2] Input : data/input/Amazon Sale Report.csv
[Task2-2] Output : data/output/Task_2-2.parquet
[Task2-2] Parsed rows: 121180
[Task2-2] Running Approach 1: approx_percentile ...
[Task2-2] Running Approach 2: exact_percentile ...
[Task2-2] Writing output ...
[Task2-2] Done. Total (SKU, month) groups: 16345
[Task2-2] Output written to: data/output/Task_2-2.parquet
[Task2-2] Sample output (top 20 rows):
```

SKU	month	p90_stddev_approx	p90_stddev_exact	p80_stddev_approx	p80_stddev_exact
AN201-RED-M	2022-04	0.0	0.0	0.0	0.0
AN201-RED-M	2022-05	0.0	0.0	0.0	0.0
AN201-RED-XL	2022-06	0.0	0.0	0.0	0.0
AN201-RED-XXL	2022-05	0.0	0.0	0.0	0.0
AN202-ORANGE-M	2022-04	0.0	0.0	0.0	0.0
AN202-ORANGE-M	2022-05	0.0	0.0	0.0	0.0
AN202-ORANGE-S	2022-04	0.0	0.0	0.0	0.0
AN202-ORANGE-S	2022-05	0.0	0.0	0.0	0.0
AN202-ORANGE-XXL	2022-04	0.0	0.0	0.0	0.0
AN203-MAROON-XXL	2022-05	0.0	0.0	0.0	0.0
AN204-PURPLE-L	2022-05	0.0	0.0	0.0	0.0
AN204-PURPLE-L	2022-06	0.0	0.0	0.0	0.0
AN204-PURPLE-M	2022-06	0.0	0.0	0.0	0.0
AN204-PURPLE-S	2022-06	0.0	0.0	0.0	0.0
AN204-PURPLE-XL	2022-05	0.0	0.0	0.0	0.0
AN204-PURPLE-XXL	2022-06	0.0	0.0	0.0	0.0
AN205-YELLOW-S	2022-04	0.0	0.0	0.0	0.0
AN205-YELLOW-XL	2022-05	0.0	0.0	0.0	0.0
AN205-YELLOW-XL	2022-06	0.0	0.0	0.0	0.0
AN205-YELLOW-XXL	2022-06	0.0	0.0	0.0	0.0

```
only showing top 20 rows

[pearspringmind@master Lab03_BigData]$
```

Hình 10: Kết quả chạy chương trình Task 2.2

```
=====
TASK 2.2 SAMPLE WITH NON-ZERO STANDARD DEVIATION VALUES
=====
```

SKU	month	p90_stddev_approx	p90_stddev_exact	p80_stddev_approx	p80_stddev_exact	
AN209-BIEGE-XL	2022-04	6.0	6.0	6.0	6.0	
BL013-62BLACK	2022-05	1.7496	1.7496	1.7496	1.7496	
BL021-71BLACK-B	2022-04	9.4045	9.4045	9.4045	9.4045	
BL024-74RED-B	2022-05	2.5	2.5	2.5	2.5	
BL035-1616OLD	2022-04	372.0	372.0	372.0	372.0	
BL098-S	2022-06	35.5	35.5	35.5	35.5	
BL110-L	2022-06	53.19	53.19	53.19	53.19	
BL113-S	2022-04	213.4753	213.4753	213.4753	213.4753	
BTM042-PP-XL	2022-04	8.0	8.0	8.0	8.0	
CH213-BROWN-S	2022-06	150.5	150.5	150.5	150.5	
J0002-SKD-L	2022-05	424.4278	424.4278	424.4278	424.4278	
J0002-SKD-M	2022-04	22.1599	22.1599	22.1599	22.1599	
J0002-SKD-M	2022-05	5.0	5.0	29.5836	29.5836	
J0002-SKD-XL	2022-05	32.4875	32.4875	32.4875	32.4875	
J0002-SKD-XL	2022-06	14.9943	14.9943	14.9943	14.9943	
J0002-SKD-XS	2022-05	33.4697	33.4697	33.4697	33.4697	
J0002-SKD-XXL	2022-05	593.0	593.0	343.1485	343.1485	
J0003-SET-L	2022-04	2.2111	2.2111	1.8325	1.8325	
J0003-SET-L	2022-05	19.3276	19.3276	15.2798	15.2798	
J0003-SET-S	2022-04	10.1666	10.1666	8.3927	8.3927	

```
only showing top 20 rows
```

Hình 11: Kết quả đo hiệu năng thời gian chạy giữa hai phương pháp

```
=====
TASK 2.2 PARQUET OUTPUT PREVIEW (Task_2-2.parquet)
=====
```

SKU	month	p90_stddev_approx	p90_stddev_exact	p80_stddev_approx	p80_stddev_exact
AN201-RED-M	2022-04	0.0	0.0	0.0	0.0
AN201-RED-M	2022-05	0.0	0.0	0.0	0.0
AN201-RED-XL	2022-06	0.0	0.0	0.0	0.0
AN201-RED-XXL	2022-05	0.0	0.0	0.0	0.0
AN202-ORANGE-M	2022-04	0.0	0.0	0.0	0.0
AN202-ORANGE-M	2022-05	0.0	0.0	0.0	0.0
AN202-ORANGE-S	2022-04	0.0	0.0	0.0	0.0
AN202-ORANGE-S	2022-05	0.0	0.0	0.0	0.0
AN202-ORANGE-XXL	2022-04	0.0	0.0	0.0	0.0
AN203-MAROON-XXL	2022-05	0.0	0.0	0.0	0.0
AN204-PURPLE-L	2022-05	0.0	0.0	0.0	0.0
AN204-PURPLE-L	2022-06	0.0	0.0	0.0	0.0
AN204-PURPLE-M	2022-06	0.0	0.0	0.0	0.0
AN204-PURPLE-S	2022-06	0.0	0.0	0.0	0.0
AN204-PURPLE-XL	2022-05	0.0	0.0	0.0	0.0
AN204-PURPLE-XXL	2022-06	0.0	0.0	0.0	0.0
AN205-YELLOW-S	2022-04	0.0	0.0	0.0	0.0
AN205-YELLOW-XL	2022-05	0.0	0.0	0.0	0.0
AN205-YELLOW-XL	2022-06	0.0	0.0	0.0	0.0
AN205-YELLOW-XXL	2022-06	0.0	0.0	0.0	0.0

```
only showing top 20 rows
```

Hình 12: Một phần nội dung tập Parquet kết quả đầu ra Task_2-2.parquet

7 Kết luận

7.1 Tổng kết kết quả

Nhóm nghiên cứu đã hoàn thành xuất sắc cả bốn nhiệm vụ được giao của Lab 03, tạo ra bốn tập dữ liệu kết quả đầu ra đúng theo các định dạng và tiêu chuẩn kỹ thuật yêu cầu:

- **Task 1-1:** Task_1-1.csv — Gồm 3,543 dòng kết quả thực tế, xác định kích cỡ được mua nhiều nhất theo mô hình cửa sổ trượt 7 ngày cho từng bang.
- **Task 1-2:** Task_1-2.csv — Gồm 128 dòng kết quả, tính toán giá trị trung vị của độ đa dạng sản phẩm cấp bang theo tháng cho các kiểu dáng có kích cỡ từ XXL trở lên.
- **Task 2-1:** Task_2-1.parquet — Gồm 2,452 dòng kết quả, tính toán tỷ lệ đơn hàng bị hủy theo từng thành phố trên tập hợp các đơn hàng thỏa mãn đồng thời ba điều kiện lọc của đề

bài.

- **Task 2-2: Task_2-2.parquet** — Tính toán độ lệch chuẩn tổng thể của giá trị đơn hàng Amount theo cặp SKU-tháng với các ngưỡng phân vị động P90/P80, đối chiếu song song giữa hai phương pháp gần đúng và chính xác.

Mỗi nhiệm vụ đều được tiến hành kiểm chứng thủ công nghiêm ngặt bằng cách chọn ngẫu nhiên các mẫu từ kết quả đầu ra, đối chiếu ngược lại với tập dữ liệu gốc và thực hiện tính toán độc lập để đảm bảo độ chính xác tuyệt đối.

7.2 Trade-off giữa RDD MapReduce và Spark Structured APIs

Tiêu chí	MapReduce RDD (Task 1-x)	Spark Structured API (Task 2-x)
Mức độ trừu tượng	Thấp — lập trình viên tự kiểm soát cặp khóa - giá trị, logic phát dữ liệu, bộ giảm.	Cao — các phép biến đổi mang tính khai báo, Catalyst tự động tối ưu hóa kế hoạch thực thi.
Độ dễ dàng triển khai	Mã nguồn dài hơn nhưng luồng xử lý cực kỳ tường minh đối với các phép toán cơ bản.	Mã nguồn ngắn gọn hơn nhiều cho các logic phức tạp; các phép liên kết/lọc/gom nhóm chỉ mất vài dòng.
Tối ưu hóa hiệu năng	Thủ công: lập trình viên tự chọn thiết kế khóa, quyết định sử dụng <code>reduceByKey</code> hay <code>groupByKey</code> .	Tự động: Catalyst tự chọn chiến lược liên kết tối ưu, cơ chế AQE tự động điều chỉnh phân vùng trong lúc chạy.
Sự phù hợp bài toán	Các bài toán cửa sổ trượt, số trung vị — cần kiểm soát chi tiết mô hình phát dữ liệu và logic sắp xếp.	Bộ lọc đa điều kiện, liên kết với giá trị trung bình của bang, tính phân vị — cần kết hợp nhiều phép biến đổi phức tạp.

Bảng 18: So sánh các tiêu chí kỹ thuật và hiệu quả lập trình giữa MapReduce RDD và Spark Structured APIs

7.3 Bài học rút ra

1. **Cửa sổ trượt $\leftrightarrow$ mô hình phát dữ liệu:** Thay vì quét ngược về quá khứ cho mỗi ngày tính toán, phương pháp “đẩy xuôi” (phát dữ liệu đóng góp cho các ngày tương lai) biến bài toán cửa sổ trượt thành bài toán gom nhóm khóa - giá trị thông thường, hoàn toàn phù hợp với mô hình MapReduce.
2. **Trung vị đòi hỏi sắp xếp toàn bộ dữ liệu:** Không thể sử dụng Combiner hay `reduceByKey` cho phép tính trung vị vì phép toán này không có tính chất kết hợp. Bài học: xác định rõ tính chất toán học của phép gom nhóm trước khi lựa chọn API xử lý.
3. **Liên kết phát tán cho bảng nhỏ:** Trong Task 2-1, bảng trung bình của bang nhỏ đủ điều kiện phát tán, giúp tránh hoàn toàn chi phí shuffle trên bảng đơn hàng lớn. Nguyên tắc: luôn kiểm tra kích thước bảng liên kết để chọn chiến lược tối ưu nhất.
4. **Phân vị gần đúng so với chính xác:** Task 2-2 chỉ ra phép tính phân vị gần đúng (thuật toán Greenwald-Khanna) cho tốc độ chạy nhanh hơn nhưng có thể cho ngưỡng khác so với chính xác (đặc biệt trên các nhóm nhỏ hoặc phân bố bị lệch dữ liệu). Phân vị chính xác qua hàm cửa sổ đảm bảo tính chính xác tuyệt đối nhưng tiêu tốn nhiều chi phí shuffle và sắp xếp dữ liệu hơn.
5. **Chuẩn hóa dữ liệu là bắt buộc:** Dữ liệu thực tế luôn chứa nhiều lỗi phát sinh như khoảng trắng thừa, kiểu chữ hoa/thường không nhất quán, các giá trị null hay định dạng ngày tháng đa dạng. Việc loại bỏ khoảng trắng và chuyển chữ hoa/thường ngay từ đầu quy trình xử lý là bắt buộc để tránh sai lệch kết quả tính toán.

7.4 Khó khăn và hướng cải thiện

Khó khăn gặp phải:

- Phân tích cú pháp ngày tháng: tập dữ liệu chứa nhiều định dạng ngày khác nhau, cần kiểm tra tuần tự qua nhiều mẫu định dạng ngày để tránh làm mất dữ liệu.
- Mã khuyến mãi: chuỗi khuyến mãi phân tách bằng dấu phẩy có khoảng trắng không đều, dấu phẩy dư thừa ở cuối tạo ra các giá trị rỗng cần lọc bỏ.

- Cấu hình Spark trên Linux: cần đồng bộ chính xác phiên bản Spark, Scala, Java và cấu hình driver bind ổn định ở chế độ chạy cục bộ.

Hướng cải thiện:

- Bổ sung bộ kiểm thử tự động (unit test) cho từng phép biến đổi dữ liệu, đối chiếu trực tiếp kết quả đầu ra với các giá trị kỳ vọng.
- Chuẩn hóa cấu trúc dữ liệu đầu vào bằng một bước tiền xử lý duy nhất cho cả bốn nhiệm vụ.
- Tối ưu hóa số lượng phân vùng dựa trên kích thước thực tế của tập dữ liệu thay vì sử dụng cấu hình mặc định.

8 Phụ lục

8.1 Source code và Demo

- Link Dataset/Output: [Drive](#)
- Source code được nộp trực tiếp trong thư mục `src` của gói bài làm.